

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Lehrstuhl für Informationstechnik
(Schwerpunkt Kommunikationselektronik)**

LIKE

Bachelor Thesis on the topic of:

Hybrid Bluetooth and mioty Localization System

Author	Esteban Ortiz Vicente
Matriculation No.	23322252
Course of Study	Computational Engineering
Supervisor	Prof. Dr.-Ing. Albert Heuberger Prof. Dr.-Ing. Jörg Robert
Registration Date	19 June 2024
Submission Date	18 September 2024

Declaration of Authorship

I declare that the work is entirely my own and was produced with no assistance from third parties.

I certify that the work has not been submitted in the same or any similar form for assessment to any other examining body and all references, direct and indirect, are indicated as such and have been cited accordingly.

Erlangen, September 18, 2024



Acknowledgement

I would like to express my deepest gratitude to my family. First, to my mother for teaching me that, even when things don't go as planned, we must always keep moving forward. To my sister, for showing me that with effort, dedication, and attention to detail, great things can be achieved. To my great-uncles, who have always been like grandparents to me, for their care and unconditional love, especially during my childhood. I also want to sincerely thank my friends for teaching me that there is more to life than just being a student, and for their companionship and support, which have made this journey much more enjoyable and manageable.

Finally, I want to express my gratitude to my thesis advisor, Prof. Dr.-Ing. Jörg Robert, for his valuable guidance and support throughout this process. Thank you.

Topic and Task of this Thesis

This Bachelor thesis aims to design and implement a Bluetooth stack enabling BLE beacon transmission via the SX1280, focusing on the integration of short and long-range communication technologies for effective asset tracking. The project involves the following key steps:

- **Literature research on Bluetooth beacons:** Conduct an extensive review of existing literature to understand the current state of Bluetooth beacon technology, including its applications, advantages, and limitations.
- **Overview of commercial systems for Bluetooth asset tracking:** Analyze existing commercial systems that utilize Bluetooth for asset tracking to identify industry standards and common practices.
- **Selection and Implementation of a Bluetooth stack:** Select an appropriate Bluetooth stack and implement it to support BLE beacon transmission on the SX1280 transceiver.
- **Integration of mioty technology:** Integrate mioty technology with BLE to provide a comprehensive solution that combines short-range and long-range communication capabilities.
- **Development of a demonstrator:** Develop a demonstrator to showcase the functionality and effectiveness of the integrated system, providing a practical example of its application.

The outcomes of this project will be documented in a written report. Additionally, the developed source code will be well-annotated with comments to facilitate understanding and future work. The project will adhere to the guidelines established by the DFG (German Research Foundation) regarding Good Scientific Practice throughout the course of this thesis.

Abstract

The LIKE initiative is advancing asset localization precision, leveraging technologies akin to the Apple Air Tag which use Bluetooth Low Energy (BLE) signals to emit Bluetooth Beacons periodically. Asset tracking is achieved through BLE receivers, such as smartphones, assuming the tracking tag's location aligns closely with the smartphone's. However, Bluetooth's limited range can be restrictive, necessitating alternative long-range solutions like mioty for broader coverage.

For cost-effective and versatile applications, it's pivotal to utilize a single transceiver hardware for both Bluetooth and mioty communications. The Semtech SX1280 transceiver, supporting LoRa, mioty, and BLE in the 2.4 GHz band, are prime candidates. Yet, the absence of a compatible Bluetooth stack for beacon transmission on these chips poses a challenge.

The Bachelor thesis aims to design and implement a Bluetooth stack enabling BLE beacon transmission via the SX1280, addressing the integration of short and long-range communication technologies for effective asset tracking.

Abstract

Die LIKE-Initiative verbessert die Präzision der Asset-Lokalisierung, indem sie Technologien ähnlich dem Apple Air Tag nutzt, die Bluetooth Low Energy (BLE)-Signale verwenden, um periodisch Bluetooth-Beacons zu senden. Das Asset-Tracking wird durch BLE-Empfänger wie Smartphones erreicht, vorausgesetzt, der Standort des Tracking-Tags stimmt eng mit dem des Smartphones überein. Allerdings kann die begrenzte Reichweite von Bluetooth einschränkend sein, was alternative Langstreckenlösungen wie mioty für eine größere Abdeckung erforderlich macht.

Für kosteneffiziente und vielseitige Anwendungen ist es entscheidend, eine einzige Transceiver-Hardware sowohl für Bluetooth als auch für mioty-Kommunikation zu nutzen. Die Semtech SX1280 Transceiver, die LoRa, mioty und BLE im 2,4 GHz-Band unterstützen, sind dafür ideale Kandidaten. Doch das Fehlen eines kompatiblen Bluetooth-Stacks für die Beacon-Übertragung auf diesen Chips stellt eine Herausforderung dar.

Die Bachelorarbeit zielt darauf ab, einen Bluetooth-Stack zu entwerfen und zu implementieren, der die BLE-Beacon-Übertragung über den SX1280 ermöglicht und die Integration von Kurz- und Langstreckenkommunikationstechnologien für effektives Asset-Tracking adressiert.

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Objectives	2
2	Theoretical Background	3
2.1	Bluetooth Low Energy (BLE) Beacons	3
2.1.1	Technical Overview of BLE Beacons	4
2.1.2	Applications of BLE Beacons: Apple AirTag Case Study	7
2.2	mioty Technology	9
2.2.1	Technical Overview of mioty	9
2.2.2	Applications of mioty in Asset Tracking	10
2.3	Semtech SX1280 Transceiver Hardware	11
2.3.1	Overview of Semtech SX1280	11
2.3.2	Suitability for Hybrid Systems	11
3	System Design Strategy	12
3.1	Conceptual Framework	12
3.2	Hardware Components	16
3.3	Software Components	17
4	Implementation of the Localization System	18
4.1	Implementation of the BLE Stack for Beacon Transmission	18
4.1.1	Overview of BLE Stack Development	18
4.1.2	Development Process	20
4.1.3	Code Details of BLE Stack for Beacon Transmission	21
4.2	Integration of BLE and mioty	32
4.2.1	System Integration Plan	32
4.2.2	Implementation Steps	34
4.3	Development and Integration of a Web Application for System Management	41
4.3.1	Overview of Web Application Development	42
4.3.2	Development Process	44
4.4	Testing and Validation	46

5	Results and Analysis	49
5.1	System Implementation Results	49
6	Conclusion and Future Work	58
6.1	Key Findings	58
6.2	Interpretation of Results	59
6.3	Limitations of the Study	59
6.4	Recommendations for Future Work	59
6.5	Implications for the Field	60
6.6	Final Thoughts	60
	List of Figures	61
	Bibliography	63

1 Introduction

This chapter outlines the motivation and context for the integration of Bluetooth Low Energy (BLE) and mioty technologies in asset tracking applications. It presents the limitations of existing BLE-based tracking solutions and the need for combining BLE with long-range communication systems like mioty. The chapter defines the primary challenges in achieving this integration, particularly the lack of a BLE stack for the SX1280 transceiver, and sets the foundation for the objectives of this thesis.

1.1 Problem Statement

Asset tracking is a critical function in various industries, from logistics to healthcare, where precise localization of assets can significantly improve operational efficiency and reduce costs [1]. The LIKE initiative is enhancing the precision of asset localization by leveraging Bluetooth Low Energy (BLE) technology, similar to that used in devices like the Apple Air Tag [2]. BLE signals are used to periodically transmit Bluetooth beacons, and asset tracking is achieved through BLE receivers, such as smartphones. This approach assumes that the location of the tracking tag is closely aligned with the location of the smartphone.

However, the reliance on Bluetooth technology presents a limitation due to its relatively short range, which restricts its use to environments where assets are always within close proximity to BLE receivers [3]. To overcome this limitation and enable asset tracking over broader areas, long-range communication technologies such as mioty are required. Mioty provides extended coverage beyond the capabilities of BLE, making it a suitable candidate for large-scale asset tracking applications [4].

The challenge is to implement a cost-effective solution that integrates short-range and long-range communication. Using separate transceivers for BLE and mioty increases hardware complexity and cost. The Semtech SX1280 transceiver, supporting LoRa, mioty, and BLE in the 2.4 GHz band, is ideal for this integration. However, there is no available Bluetooth stack for BLE beacon transmission on this chip, which this thesis seeks to address

The primary problem this thesis seeks to solve is the absence of a Bluetooth stack for the SX1280 transceiver that enables BLE beacon transmission. By developing this capability, the thesis will facilitate the integration of short-range BLE and long-range mioty communications, offering a comprehensive solution for effective and scalable asset tracking.

1.2 Objectives

The objective of this thesis is to design and implement a Bluetooth stack for the SX1280 transceiver to enable BLE beacon transmission. This involves integrating BLE with mioty technology to create a robust asset-tracking solution that combines short-range and long-range communication. Key tasks include reviewing existing literature, analyzing commercial systems, selecting and implementing the Bluetooth stack, developing a demonstrator, and documenting the project for future reference.

2 Theoretical Background

This chapter provides an overview of the key technologies and hardware involved in the integration of Bluetooth Low Energy (BLE) and mioty for asset tracking. It begins by explaining BLE beacons, focusing on their low-power, short-range communication, data rates, and packet structures, followed by a practical case study of Apple’s AirTag system. The discussion then moves to mioty, a long-range, low-power wide-area network (LPWAN) protocol, highlighting its network architecture, data transmission techniques, and Telegram Splitting (TS-UNB) method for improved reliability. Finally, the chapter introduces the Semtech SX1280 transceiver as the ideal hardware platform for combining BLE and mioty within a single device, due to its ability to support both short-range and long-range communication through multiple modulation schemes. This foundational knowledge will support the implementation of the hybrid BLE-mioty system presented in the thesis.

2.1 Bluetooth Low Energy (BLE) Beacons

Bluetooth Low Energy (BLE) is a wireless technology designed for applications that require low power consumption, making it ideal for battery-powered devices [3]. BLE devices can operate in various roles, including Broadcaster, Observer, Peripheral, and Central. The Broadcaster role is particularly crucial for beacon applications, where devices send out advertising packets on specific channels without needing to establish a connection [5]. This method enables efficient, one-way communication, perfect for use cases such as location tracking and proximity-based services. BLE beacons, operating as Broadcasters, periodically transmit small data packets using BLE advertising channels. These packets can contain universally unique identifiers (UUIDs), URLs, or sensor data, tailored for specific applications [6]. The ability of BLE beacons to function without a continuous connection, combined with their adjustable transmission ranges and low power requirements, makes them highly energy-efficient and well-suited for long-term deployment.

This section summarizes the key technical aspects of BLE for beacon transmission, including data rates, range, frequency bands, modulation, error-checking, and data whitening, while also exploring practical applications like Apple’s asset tracking system.

2.1.1 Technical Overview of BLE Beacons

Data Rate and Transmission Range

BLE offers various data rates to accommodate different application requirements. BLE 4.0 provides a base data rate of 1 Mbps, while BLE 5.0 introduces an option of 2 Mbps for faster data transmission. Regarding transmission range, typical operating distances for BLE range between 10 to 100 meters. The extended range capabilities of BLE 5.0 further enhance its utility in both short and long-range applications [7].

Frequency Band and Channels

Bluetooth Low Energy (BLE) operates within the globally available 2.4 GHz ISM band, which is segmented into 40 channels, each 2 MHz wide [8]. As depicted in Figure 2.1, three of these channels (37, 38, and 39) are specifically allocated for advertising. These channels are strategically positioned to minimize interference from other wireless technologies, such as Wi-Fi. The remaining 37 channels (0-36) are used for data transmission. To further enhance communication reliability and reduce interference, BLE employs Frequency Hopping Spread Spectrum (FHSS) across the data channels, dynamically switching frequencies during a communication session [5].

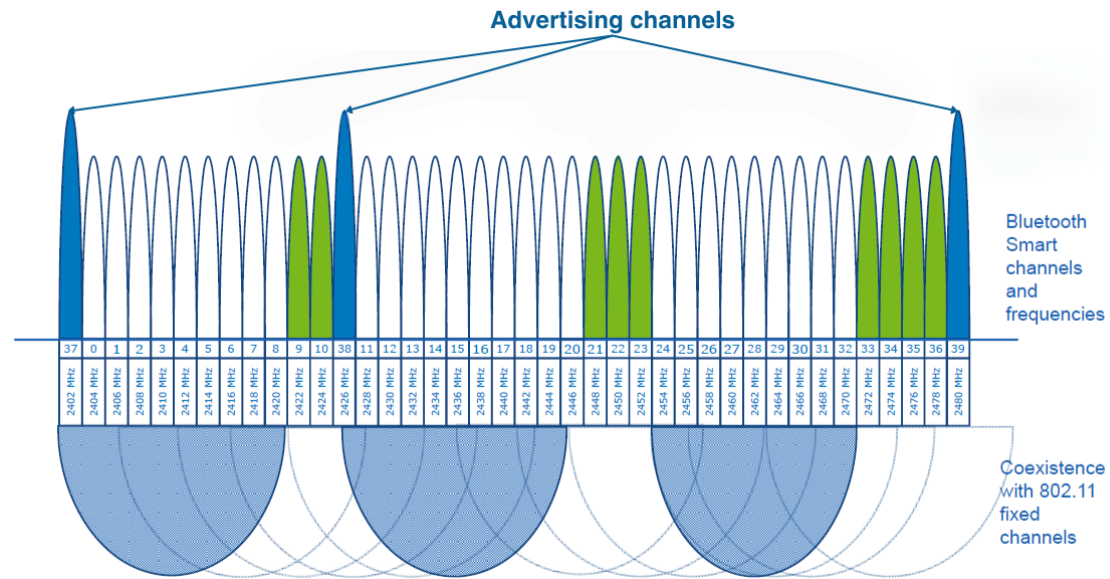


Figure 2.1: BLE spectrum diagram showing the distribution of advertising and data channels. Adapted from [8], with modifications.

2.1. BLUETOOTH LOW ENERGY (BLE) BEACONS

Modulation Scheme

BLE uses Gaussian Frequency-Shift Keying (GFSK) as its modulation scheme, which smooths the transitions of the carrier signal by applying a Gaussian filter before the modulator. This helps reduce the signal's bandwidth, minimizing interference with adjacent channels [9].

Link Layer Packet Format

The structure of BLE advertising packets includes several fields, as illustrated in Figure 2.2, these fields are organized to ensure reliable data transmission and adherence to BLE specifications [7].

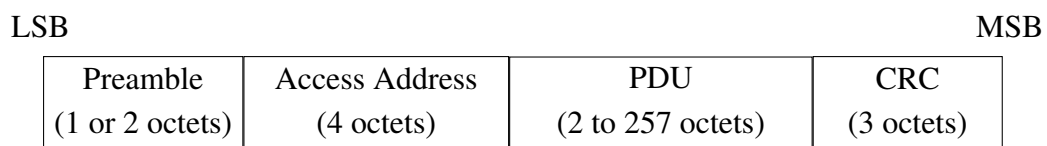


Figure 2.2: Link Layer Packet Format in BLE. Adapted from [7], with modifications.

- **Preamble (1 or 2 octets):** The preamble is a sequence of alternating bits (0 and 1) used for synchronization and training at the receiver. It has 8 bits for the LE 1M PHY channel (1 Mbps) and 16 bits for the LE 2M PHY channel (2 Mbps). The first bit matches the LSB of the Access Address [7].
- **Access Address (4 octets):** A unique address used to identify and filter specific packets in periodic advertising channels. For other advertising physical channels, a fixed value of 0x8E89BED6 is used [7].
- **Protocol Data Unit (PDU) (2 to 257 octets):** Contains the payload data, which may include device identification, sensor data, or location information [6].
- **Cyclic Redundancy Check (CRC) (3 octets):** A 24-bit CRC is calculated over the PDU to detect errors and ensure data integrity during transmission [10].

2.1. BLUETOOTH LOW ENERGY (BLE) BEACONS

Error Checking

To ensure transmission reliability, error checking is employed. According to the BLE specifications [10], the following 24-bit polynomial is used for CRC calculation.

$$x^{24} + x^{10} + x^9 + x^6 + x^4 + x^3 + x + 1 \quad (2.1.1)$$

CRC begins with an initialization value or "CRC seed", which varies depending on the type of PDU (e.g., 0x555555 for Advertising Physical Channel PDUs) [10]. The CRC is then computed by applying a polynomial to the bits of the PDU, starting from the least significant bit. Upon receiving the packet, the CRC is recalculated and compared with the transmitted CRC to detect errors.

Data Whitening

Data whitening utilizes a 7-bit linear feedback shift register (LFSR), defined by the following polynomial:

$$x^7 + x^4 + 1 \quad (2.1.2)$$

The LFSR is initialized with a sequence or seed, where each seed is determined by the channel index being used for transmission [10]. This helps prevent long sequences of zeros or ones that could cause synchronization issues.

BLE Advertising Formats: iBeacon and Eddystone

As shown in Figure 2.3, the PDU consists of a PDU header, which specifies the packet type and length, and a PDU payload. The PDU payload contains both an advertising address (Adv Address) and advertising data (Adv Data) [7].

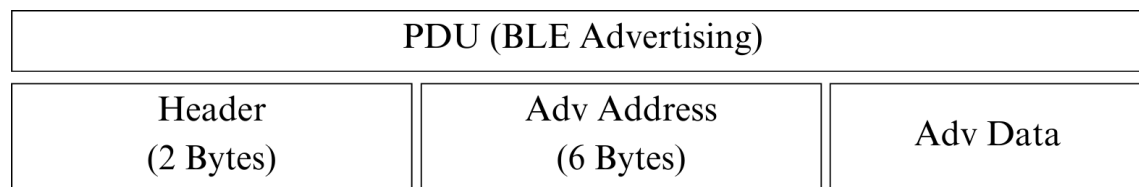


Figure 2.3: Structure of BLE PDU formats, highlighting the PDU header and payload components based on the information provided in the [7].

2.1. BLUETOOTH LOW ENERGY (BLE) BEACONS

There are several beacon protocols, with the most widely used being iBeacon, developed by Apple, and Eddystone, developed by Google [11]. Each protocol uses its own unique format, as shown in Figures 2.4 y 2.5.

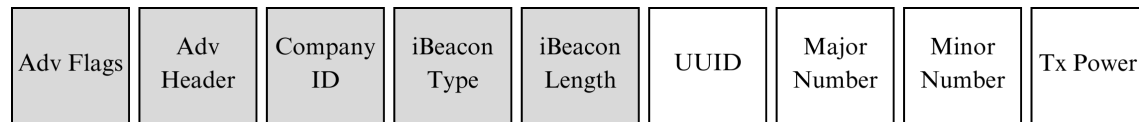


Figure 2.4: iBeacon format showing the structure of the UUID is based on the information provided in the [12].

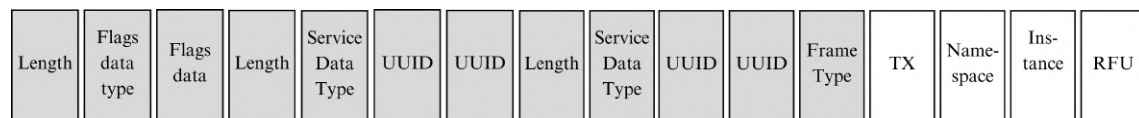


Figure 2.5: Eddystone UID format showing the structure used for broadcasting unique identifiers for asset tracking based on the Eddystone UID format, as described in the [13].

2.1.2 Applications of BLE Beacons: Apple AirTag Case Study

Apple AirTag is a small tracking device developed by Apple Inc. and released in 2021 [14]. This device allows users to locate personal items using Apple’s global network of devices.

How Apple AirTag Utilizes BLE

Apple AirTag uses Bluetooth Low Energy (BLE) technology to periodically broadcast signals detectable by nearby iOS devices, like iPhones and iPads. These devices act as nodes in Apple’s “Find My” network. Once a signal is detected, the iOS device anonymously relays the AirTag’s location to Apple’s servers over the internet. This information is encrypted and sent to the AirTag owner’s iCloud account, ensuring privacy for both the owner and the user of the detecting device. The owner can then view the AirTag’s last known location on a map in the “Find My” app [2]. The Figure 2.6 aims to explain this functionality.

2.1. BLUETOOTH LOW ENERGY (BLE) BEACONS

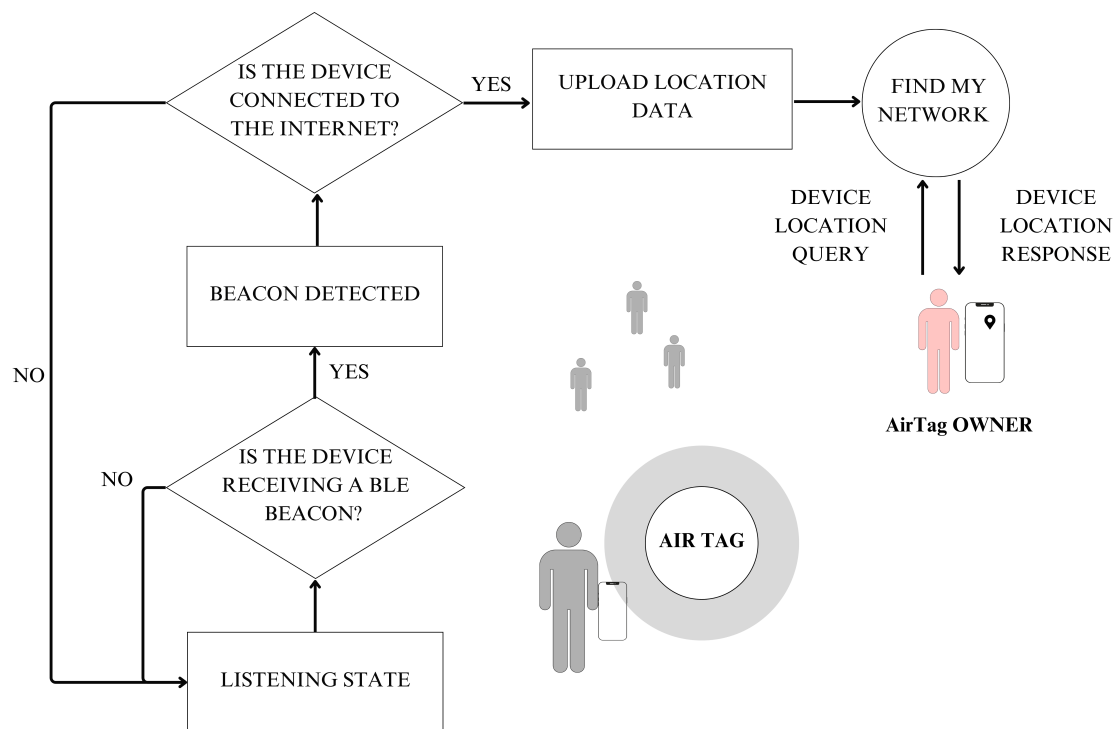


Figure 2.6: Flowchart illustrating the operation of Apple AirTag within the Find My network based on the information provided in the [14].

Advantages of Apple AirTag:

Apple's main advantage is its extensive network of devices that can act as nodes. The more devices registered in the "Find My" network, the higher the chances of detecting an AirTag. Additionally, the AirTag offers high accuracy, an affordable cost, and energy efficiency.

Limitations of Apple AirTag:

The main disadvantage is that the AirTag relies solely on the presence of an iOS device in the area within BLE range, which is relatively short. This means that in areas with low node density, the AirTag may not be detected. Additionally, the device acting as a node must not only be within BLE range but also have an internet connection to upload the location to the network.

2.2 mioty Technology

mioty is an advanced low-power wide-area network (LPWAN) protocol [15]. Developed by FRAUNHOFER IIS and the mioty alliance. This section examines mioty's technical aspects, including its data rate, transmission range, innovative TSMA and UNB communication methods, network architecture, and its application in asset tracking.

2.2.1 Technical Overview of mioty

Network Architecture

The architecture of mioty follows a star model, where a central gateway collects the data transmitted by the end nodes [4].

Data Rate and Transmission Range

mioty is optimized for transmitting short packets. With 10-byte packets, it can transmit 99 messages per hour with an airtime of 363 ms. For larger packets, such as 50 and 200 bytes, transmission rates decrease, reaching 11 messages per hour in the latter case. In terms of range, mioty offers extensive coverage, reaching up to 15 km in flat terrain and 5 km in urban areas [4].

Concept of Telegram Splitting

mioty uses Ultra Narrow Band (UNB) technology, meaning it employs narrow channels, which allows for increased transmission distances. However, this also leads to longer transmission times, making communication more susceptible to interference. To mitigate this issue, mioty employs additional techniques, such as Telegram Splitting Ultra Narrow Band (TS-UNB), which splits the packets into sub-packets and transmits them across different frequencies following a specific pattern, reducing the impact of interference [4].

TS-UNB Uplink Frame Structure

For the TS-UNB uplink, a telegram is structured with a fixed length of 186 bits and can carry up to 160 bits of payload. After channel coding and interleaving, the telegram is divided into 24 sub-packets [4]. The *core frame* consists of 24 radio bursts that carry the information of the shortest telegram. To send additional data, an *extension frame* is used, as shown in Figure 2.7 .

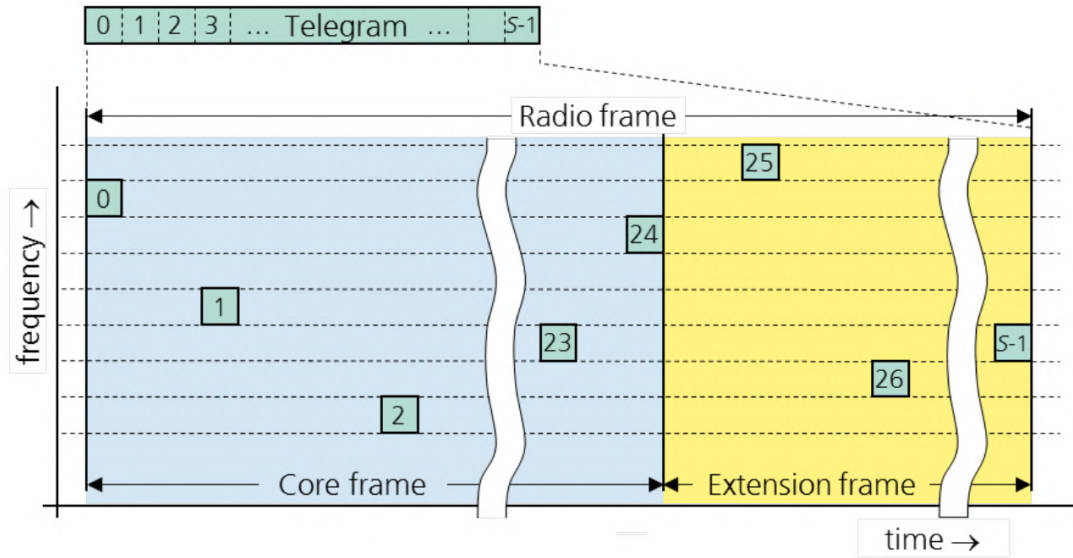


Figure 2.7: Subdivision of an uplink radio frame into a core frame and an extension frame from [4].

The TS-UNB uplink frame structure, Figure 2.8, supports up to 255 bytes, with a variable-length MPDU, zero padding, CRCs for error detection, and a Packet Size Indicator (PSI).

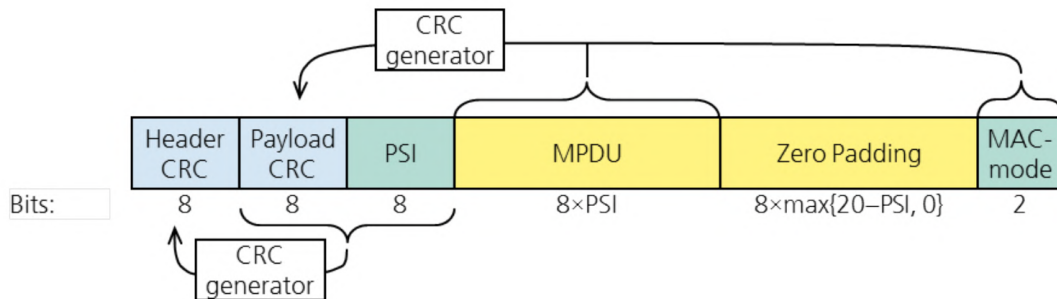


Figure 2.8: Uplink payload structure on the physical layer from [4].

2.2.2 Applications of mioty in Asset Tracking

mioty is ideal for asset tracking due to its extensive coverage area reaching kilometers, excellent scalability for large deployments, and strong interference resistance [15], ensuring reliable transmission of crucial asset location data.

2.3 Semtech SX1280 Transceiver Hardware

The SX1280 is a 2.4GHz ISM band transceiver designed for long-range wireless applications with low power consumption [16]. This section will analyze its modulation capabilities, energy efficiency, regulatory compliance, interface with host microcontrollers, and why it is perfect for a hybrid system.

2.3.1 Overview of Semtech SX1280

The SX1280 supports multiple modulations, including LoRa, FLRC, and (G)FSK, making it adaptable for various wireless applications. Its energy-efficient design is perfect for low-power devices, helping extend battery life. The chip complies with global 2.4GHz communication regulations, ensuring compatibility in international markets. It easily integrates with microcontrollers through SPI or UART interfaces, allowing flexible control and configuration [16].

2.3.2 Suitability for Hybrid Systems

The SX1280 is ideal for hybrid systems, supporting mioty's MSK and GMSK modulations, as well as BLE at the physical layer [16]. Specifically, the chip enables mioty transmission by switching the transmitter into continuous transmission mode, while the microcontroller generates the symbol clock for radio bursts. This capability makes the SX1280 a versatile solution for applications requiring compatibility across different RF technologies.

3 System Design Strategy

In the previous chapter, BLE and mioty technologies were examined, along with a chip capable of transmitting both. It was identified that BLE is a short-range technology, effective between 10 and 100 meters, that enables frequent transmission of advertising packets containing for a example a unique identifier, which makes BLE particularly suitable for indoor environments where precise location tracking is required. On the other hand, mioty is a robust, interference-resistant technology that provides coverage in areas up to 10 kilometers.

Additionally, the Apple AirTag commercial localization system was reviewed, which offers precise tracking through the transmission of BLE beacons and the "Find My" network. This network facilitates location tracking by using participating nodes to relay the location when an AirTag is detected. However, this system has limitations due to the need for the device to be within BLE range, which is relatively short, and for the device to have an internet connection.

This chapter proposes a hybrid localization system that integrates the advantages of BLE and mioty technologies to address the identified limitations. The design concept, necessary hardware components, and software elements are detailed below.

3.1 Conceptual Framework

The strategy is based on the use of two types of devices: on the one hand, end-devices capable of transmitting both BLE and mioty packets, which need to be located, and on the other hand, Gateway devices that receive the packets, calculate the coordinates, and store them on a server that uses an application to visualize the location in real time. The following Figure 3.1 illustrates this concept.

3.1. CONCEPTUAL FRAMEWORK

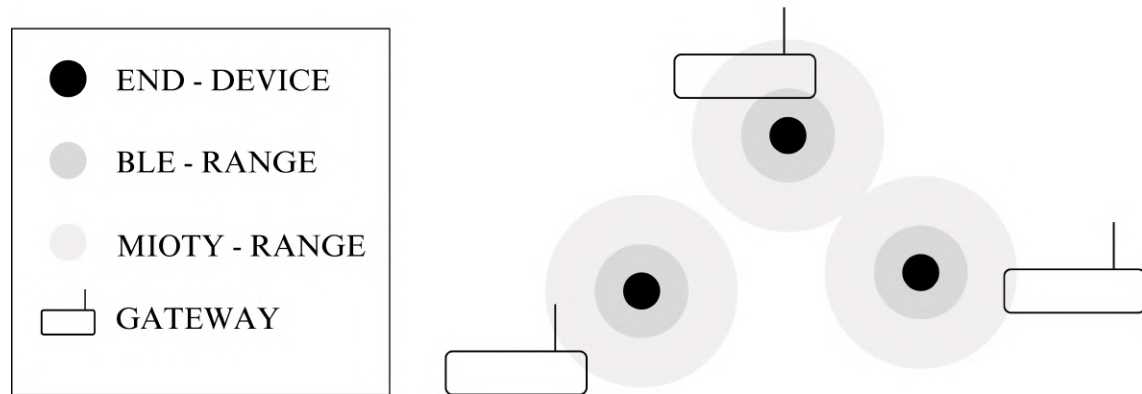


Figure 3.1: Hybrid location system with end devices, BLE range, mioty range, and gateways.

Each end device is programmed to transmit BLE packets with its unique identifier (UUID) every second for frequent short-range location updates. Additionally, the devices send mioty packets with their UUID every 15 minutes to enable long-range tracking.

These packets are received by a Gateway, which is responsible for determining the location of the device by monitoring the time of the last packet received, both from BLE and mioty. This means that if a packet has not been detected within a certain period of time, mioty technology is used, as it assumes the device is out of BLE range.

Once the technology to be used is determined, the coordinates are calculated and sent to a central server via an API request [17]. This server will be able to first communicate with a database to store the location information, and also generate a web interface to display all this information to an end user.

Figure 3.2 presents a detailed diagram of the hybrid localization system architecture, showing the specific modules each component uses for signal transmission, data processing, and storage.

3.1. CONCEPTUAL FRAMEWORK

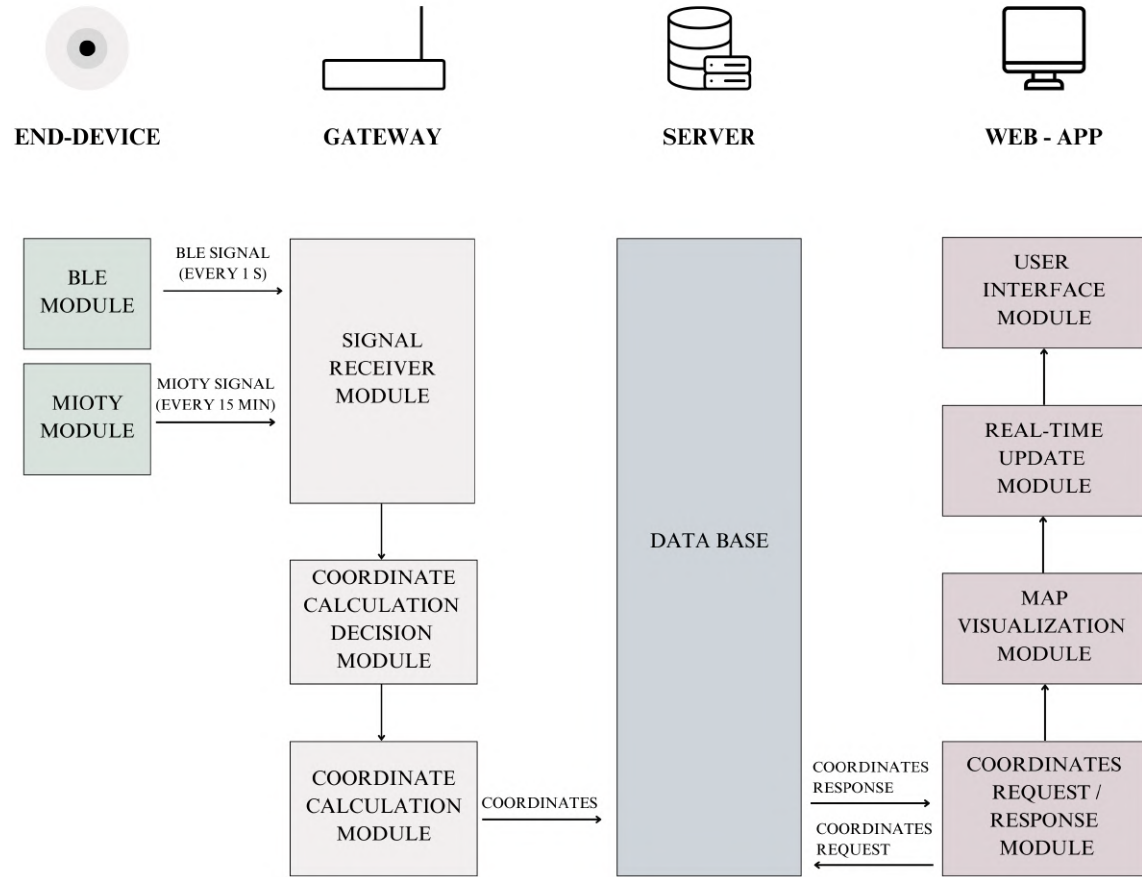


Figure 3.2: System Architecture of the Hybrid Location System Integrating BLE and mioty Technologies.

The following will detail the functionality of each module represented in the diagram, describing its purpose and how it contributes to the overall process of the hybrid localization system.

End-Device Modules

The **BLE Module** is responsible for transmitting BLE (Bluetooth Low Energy) signals that include the device's UUID every second. This module enables real-time tracking of end devices within the BLE short range. Its primary purpose is to provide frequent location updates, allowing for precise localization in areas where detailed and constant monitoring is needed.

3.1. CONCEPTUAL FRAMEWORK

The **mioty Module**, on the other hand, handles transmitting mioty signals with the device's UUID every 15 minutes to ensure location coverage over greater distances. This module aims to track device locations outside the BLE range.

Gateway Modules

Regarding the gateway modules, the **Signal Receiver Module** is the first point of contact in the gateway for any incoming communication. This module receives both BLE and mioty signals sent by the end devices. Its main function is to capture and manage all signals from end devices for further processing.

The **Coordinate Calculation Decision Module** decides which coordinate calculation method to use based on the received signals. If the BLE signal is available and within range, BLE is used, otherwise, mioty is employed. The purpose of this module is to optimize location calculation by using the most appropriate signal technology depending on the proximity of the end-device.

Finally, the **Coordinate Calculation Module** is responsible for calculating the coordinates of the end-devices based on the selected signal technology (BLE or mioty). This module uses specific algorithms to determine the precise location of the device. Its purpose is to provide the exact coordinates of the end device for accurate location monitoring.

Server Module

The **Database** stores the calculated coordinates of the end-devices and other relevant data. It serves as a central repository for all location information, maintaining an organized and accessible record of all coordinates for future queries and real-time analysis. The main purpose of this module is to ensure that location data is available and easily retrievable whenever needed.

Web-App Modules

The **Coordinates Request/Response Module** manages coordinate requests sent to the server and receives responses with updated coordinates of the end-devices. This module plays a crucial role in facilitating communication between the web application and the server, allowing for the retrieval of real-time location data and ensuring that users have access to the most current information.

The **Map Visualization Module** displays the coordinates of the end-devices on an interactive map, allowing users to see locations in real-time.

3.2. HARDWARE COMPONENTS

The **Real-Time Update Module** ensures that the map visualization is updated with the latest coordinates received from the server. By keeping the displayed information up-to-date, this module helps maintain the accuracy and timeliness of location data within the web application, allowing users to make informed decisions based on the most current information available.

The **User Interface Module** provides the user interface to interact with the web application. It allows users to request coordinates.

3.2 Hardware Components

The design employs a Raspberry Pi Pico microcontroller [18] interconnected with the SX1280 chip [16] for the end-devices, as illustrated in Fig 3.3. The Raspberry Pi Pico was chosen for its cost-effectiveness, low power consumption, and sufficient computing power to handle the BLE and mioty communication protocols.

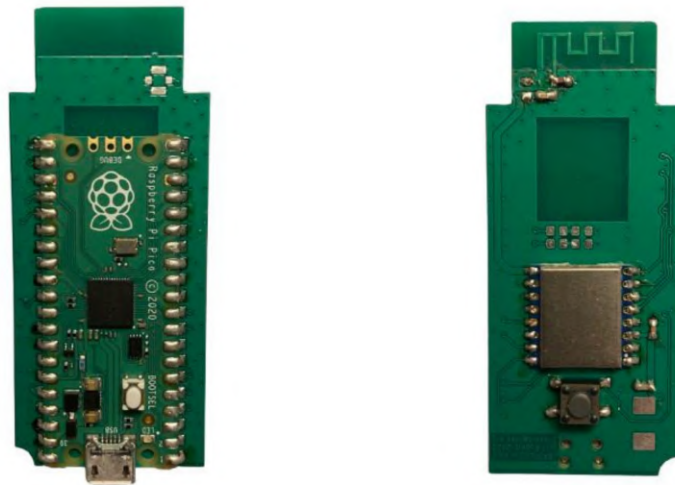


Figure 3.3: Raspberry Pi Pico microcontroller interconnected with the SX1280 chip

3.3 Software Components

This section details the software requirements for BLE, as well as for uploading coordinate data to the server and the web application interface.

The BLE and mioty transmission functionalities are implemented using the Raspberry Pi Pico framework [19], which provides a lightweight development environment suitable for embedded systems. The software includes a BLE library capable of transmitting beacons with unique identifiers (UUIDs) at regular intervals.

The API development for the server-side application is planned using the Next.js framework [20]. This choice leverages Next.js's capabilities for server-side rendering and static site generation, which enhances the performance and scalability of the web application. The API endpoints [21] will be responsible for handling coordinate data requests and responses between the gateway and the server, ensuring data integrity and security during transmission.

Additionally, the web application requires a real-time data management system to handle incoming coordinate updates from the gateway. The interface should be intuitive, allowing users to monitor asset locations, view historical data, and customize tracking settings. The software should also support data visualization tools, such as interactive maps, to enhance the user's ability to interpret and respond effectively to location data.

4 Implementation of the Localization System

This chapter describes the implementation of the localization system designed in the previous chapter. It begins with **Section 4.1: Implementation of the Bluetooth Stack**, which details the development of a Bluetooth stack capable of transmitting advertising beacons containing a UUID at one-second intervals. **Section 4.2: Integration of BLE and mioty** explains how this BLE library is integrated with the mioty software. Finally, **Section 4.3: Development and Integration of a Web Application for System Management** covers the development of an API for managing coordinate data between the gateway and the server, along with the creation and integration of a web application that allows end users to visualize coordinates and manage the system effectively.

4.1 Implementation of the BLE Stack for Beacon Transmission

4.1.1 Overview of BLE Stack Development

This section details the implementation of the Bluetooth Low Energy (BLE) stack used in the hybrid localization system. The focus is on developing a simplified BLE stack that is specifically tailored for transmitting beacons with unique identifiers (UUIDs) to support accurate location tracking. The section covers the selection criteria for the BLE stack, the layers implemented to meet the project's requirements, and the rationale for choosing a custom development approach based on existing Semtech code [22].

Definition and Purpose of the BLE Stack

A Bluetooth stack is a set of software protocols that enables wireless communication between devices using Bluetooth technology [23].

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

Layers Implemented in the BLE Stack

The BLE stack developed for this system is limited to the layers necessary for beacon transmission, excluding many layers that would be required for a full BLE stack supporting a wider range of functions (such as connection, encryption, and data transfer). The key layers implemented are:

- **Link Layer (LL):** This layer is responsible for managing advertising events and the transmission of advertising packets that contain the UUID in the beacon data field. It configures advertising parameters such as transmission intervals, advertising channels, and the content of the data field, which in this case is the unique UUID of each device. The Link Layer handles the broadcasting of these packets [24].
- **Physical Layer (PHY):** The Physical Layer is responsible for the modulation, and transmission of radio signals over the 2.4 GHz ISM band. In this simplified BLE stack, the Physical Layer is configured specifically to support the transmission of advertising packets. It does not support connection establishment or bidirectional data transfer, focusing solely on the broadcast of BLE advertising signals [24].

Selection Criteria for the Development of the BLE Stack

Instead of using a full commercial BLE stack, it was decided to develop a custom stack for this project. This decision was based on several key criteria:

1. **Specific Project Requirements:** Since the hybrid localization system only requires the ability to transmit BLE beacons, a full stack would have been unnecessary and would have added complexity and resource consumption without additional benefits.
2. **Simplicity and Efficiency:** By limiting the stack to only the layers necessary for BLE advertising, memory and power consumption were significantly reduced, which is crucial in low-power embedded devices like the Raspberry Pi Pico.
3. **Flexibility and Customization:** Developing a custom stack allowed for the customization of the advertising packet configuration, specifically to include the UUID in a defined data field. This flexibility is essential for optimizing the stack according to the specific transmission requirements of the project.
4. **Based on Existing Code:** The developed BLE stack is adapted from Semtech's Nordic-based stack [22].

4.1.2 Development Process

This section details the comprehensive process undertaken to develop and implement the Bluetooth Low Energy (BLE) stack for the hybrid localization system. The process began with the setup of the development environment and the integration of hardware components, followed by a deep dive into understanding the SX1280 chip's functionality through its datasheet [16]. Key steps included basic interaction with the chip to establish initial communication, transitioning to a class-based approach for better code management, and implementing BLE signal transmission.

Setting Up the Development Environment and Integrating Hardware Components

Visual Studio Code was used as the primary Integrated Development Environment (IDE) [25], leveraging the specific extension for the Raspberry Pi Pico provided by Raspberry Pi [26]. This extension, as described in the official guide "Getting Started with Raspberry Pi Pico"[19], simplifies the setup of the development environment by automatically managing dependencies and facilitating code compilation and debugging.

The hardware configuration involved connecting a Raspberry Pi Pico [18] to an SX1280 chip [16], following a specific wiring schematic. This setup was implemented using a custom printed circuit board (PCB).

Understanding the Functionality of the SX1280 Chip

To implement the BLE stack, it was essential to thoroughly understand the SX1280 chip's functionality by reviewing its datasheet [16]. Key sections included *Section 9: Digital Interface and Control*, which detailed the use of the SPI communication interface for controlling the chip; *Section 10: Operational Modes*, which described the various modes and state transitions that dictate the chip's behavior in different communication scenarios; *Section 11: Host Controller Interface*, which explained the command structures and procedures for accessing registers and performing data operations; and *Section 14: Transceiver Operation*, which provided step-by-step instructions for configuring the chip for BLE packet transmission. This comprehensive review ensured proper configuration and effective BLE beacon transmission.

Basic Interaction with the SX1280 Chip

To start working with the SX1280 chip, a basic code was created to establish communication and display the firmware version via serial communication, aiming to understand the initialization, command writing, register access, and data management processes.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

Transition to a Class-Based Approach

To improve code modularity and reusability, the `SX1280Hal` class was developed to encapsulate all the necessary functionalities for interacting with the SX1280 chip. This class-based approach organizes various tasks into specific methods, such as chip initialization, command exchange, register access, and radio configuration, making the code easier to navigate and maintain.

Implementation of the Necessary Methods for BLE Transmission

The development of the initial code for transmitting BLE signals began with implementing the necessary methods in the `SX1280Hal` class. These methods were specifically designed to configure the SX1280 chip according to the procedures detailed in *Section 14.2 Bluetooth Low Energy Compatible Operation* of the datasheet, which describes the operation compatible with Bluetooth Low Energy (BLE).

4.1.3 Code Details of BLE Stack for Beacon Transmission

This section delves into the details of the implementation of the BLE Stack for Beacon Transmission, which constitutes the BLE module described in Figure 3.2.

Approach

The Figure 4.1 shows the different components used in the BLE MODULE. At the bottom is the `SX1280 DRIVER` block, which manages all interactions with the SX1280 chip. This block includes functions for initializing the chip, writing commands, accessing registers, managing buffers, switching chip modes, and configuring transceiver settings. To its right, the `UTILS` block provides auxiliary functions, such as blinking the LED when a transmission occurs. At the top is the `TRANSMITTER` block, which handles parameter configuration, advertising packet definition, and channel transmission management.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

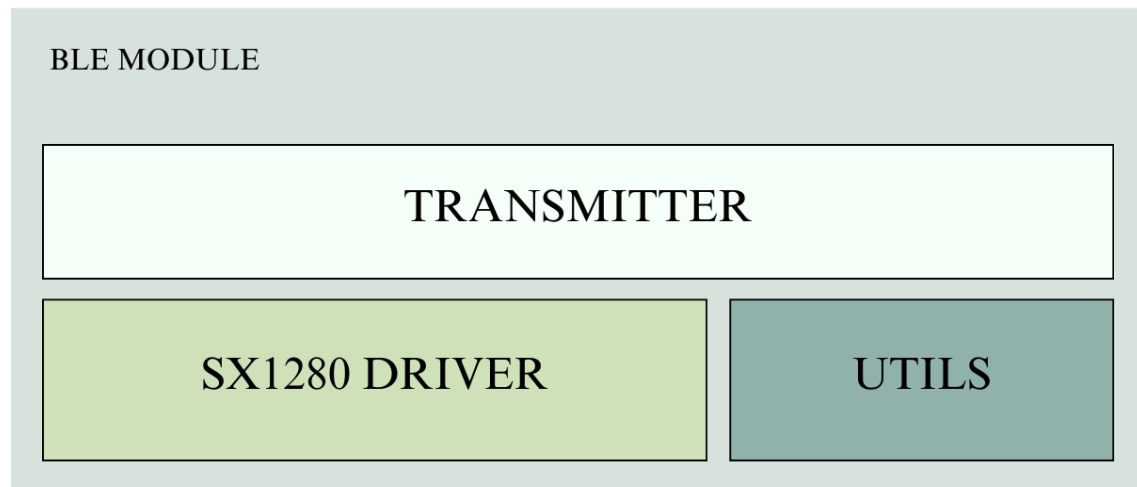


Figure 4.1: Diagram of the BLE module, showing the structure and components used.

The code structure is designed to reflect this architecture, with each block corresponding to a dedicated folder. The `SX1280Driver` folder contains the files `sx1280.cpp` and `sx1280.h`. The `Utils` folder includes `utils.cpp` and `utils.h`, while the `Transmitter` folder contains `transmitter.cpp` and `transmitter.h`.

The following sections will review the files contained within each block.

SX1280 Driver Header File: `sx1280.h`

The `sx1280.h` file defines the interface for controlling the SX1280 radio module. It provides necessary macros, enumerations, and class definitions to manage the radio's operation.

The file contains definitions for GPIO pins, register addresses, and opcodes needed for low-level hardware control, as well as enumerations for configuring the radio's modes and packet types, and data structures for managing timing and packet statuses. It sets up GPIO pins for handling interrupts from the SX1280 chip, provides register addresses and commands for direct hardware interaction, and includes an abstract class, `SX1280`, for core functions and a concrete class, `SX1280Hal`, which implements these functions to enable direct control over the radio module.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

SX1280 Driver Implementation File: `sx1280.cpp`

The `sx1280.cpp` file contains the implementation of the `SX1280Hal` class, which is responsible for communication with the SX1280 radio module via the SPI interface. This file complements the header file `sx1280.h` by providing the method definitions declared therein. It is important to highlight that the methods `Reset`, `Wakeup`, `IoIrqInit`, along with `SetRegulatorMode` and `SetStandby`, are crucial for the proper initialization of the radio. These methods ensure that the radio is in an optimal state to begin operation. Additionally, there are three key pairs of methods based on `WaitOnBusy` that are fundamental to the radio's operation. The first pair, `ReadRegister` and `WriteRegister`, enables the execution of the methods `SetBleAccessAddress`, `SetBleAdvertiserAccessAddress`, `SetCrcSeed`, and `SetWhiteningSeed`. The second pair, `ReadBuffer` and `WriteBuffer`, is essential for the `SetupAdvPdu` method, which configures the transmission packets. Finally, the third pair, `ReadCommand` and `WriteCommand`, is used for a wide range of configurations and operations, such as `GetPacketStatus`, `SetStandby`, `SetPacketType`, `SetRfFrequency`, `SetBufferBaseAddresses`, `SetModulationParams`, `SetPacketParams`, `SetTxParams`, `SetDioIrqParams`, `SetTx`, and `ClearIrqStatus`, providing comprehensive control over the radio's behavior.

Utils Header and Implementation Files: `utils.h` and `utils.cpp`

The `utils.h` and `utils.cpp` files provide essential utility functions such as `initializeLed`, `blink`, `waitUntilUsbConnected`, and `displayPacketStatus`. These functions aid in the operation and debugging of the SX1280 radio module.

Transmitter Header and Implementation Files: `transmitter.h` and `transmitter.cpp`

The `transmitter.h` and `transmitter.cpp` files manage BLE advertising packets using the SX1280 radio module. They include functions for initializing the radio, and transmitting data across BLE advertising channels. Key functions include `dio_irq_handler`, `prepareForBeacon`, `initializeRadio`, and `transmitData`, which configure the radio, manage transmission, and handle interrupts.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

Demo Program for Beacon Transmission Testing

The following example code demonstrates how to use the BLE stack for beacon transmissions, which is part of the BLE module block. This program initializes and transmits BLE (Bluetooth Low Energy) advertising beacons using the SX1280 radio module. It sets up the radio for transmission, selects a specific advertising format (iBeacon [12] in this example), and continuously broadcasts the beacon.

The code follows the diagram below Figure 4.2, which will be explained step by step.

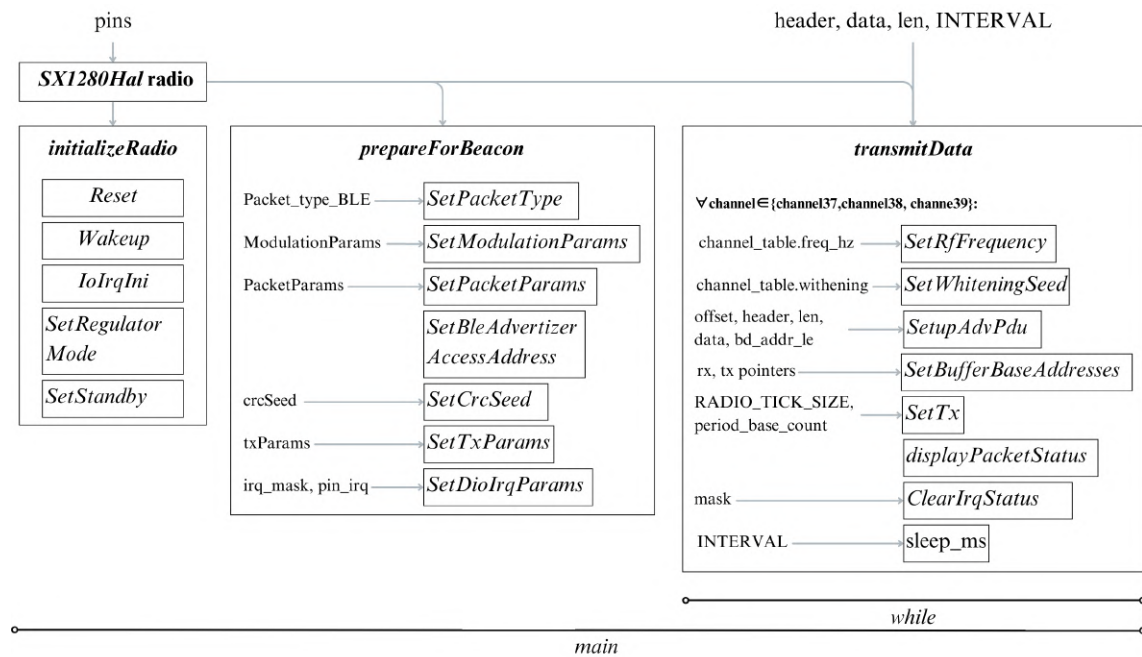


Figure 4.2: Diagram of the BLE module, showing the structure and components used.

First, the necessary header files are included to provide access to the SX1280 radio driver, utility functions, and transmitter functionality:

```

1 #include "SX1280Driver/sx1280.h"
2 #include "UTILS/utils.h"
3 #include "Transmitter/transmitter.h"
4 #include <stdio.h>
5 #include <string.h>

```

Listing 4.1: Including Required Headers

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

At the beginning of the code, the header is defined, which will be the PDU header, and data, which will be the Adv Data of the PDU. Depending on the selected protocol, either iBeacon or Eddystone, the data may vary, and its length will change accordingly.

```
1  #define INTERVAL 100
2  uint8_t header = 0x42;
3  uint8_t ibeacon_data[] = {
4      0x02,
5      0x01,
6      0x06,
7      0x1A,
8      0xFF,
9      0x4C, 0x00,
10     0x02,
11     0x15,
12     0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
13     0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10,
14     0x00, 0x01,
15     0x00, 0x02,
16     0xC5
17 };
18 uint8_t eddystone_template[] = {
19     0x02, 0x01, 0x06,
20     0x03, 0x03, 0xAA, 0xFE,
21     0x17, 0x16, 0xAA, 0xFE,
22     0x00,
23     0xF8,
24     0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09,
25     0x0A,
26     0x01, 0x02, 0x03, 0x04, 0x05, 0x06,
27     0x00, 0x00
28 };
29 uint8_t len = sizeof(ibeacon_data);
30 // uint8_t len = sizeof(eddystone_template);
```

Listing 4.2: BLE iBeacon and Eddystone UID Data Templates

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

The `main` function begins by initializing standard input and output, which allows communication over USB for debugging. It then creates an instance of the `SX1280Hal` class, specifying the pins defined for SPI communication between the Raspberry Pi Pico microcontroller and the SX1280 transceiver. Once the instance is created, the `initializeRadio` function configures the SX1280 radio module by resetting it, waking it from sleep mode, setting interrupts, and setting power and standby modes, and then the `prepareForBeacon` configures the SX1280 radio for BLE advertising.

```
1  int main()
2  {
3      stdio_init_all();
4      SX1280Hal radio(17, 7, 6, 19, 16, 18);
5      waitUntilUsbConnected();
6      initializeRadio(radio);
7      prepareForBeacon(radio);
```

Listing 4.3: Main Function Initialization

The code enters an infinite loop, continuously using the `transmitData` function to handle the transmission of the BLE advertising packet across all three BLE advertising channels (37, 38, and 39).

```
1      while (true)
2      {
3          transmitData(radio, ibeacon_data, header, len,
4                      INTERVAL);
5          //transmitData(radio, eddystone_template, header, len
6                      , INTERVAL);
7      }
8  }
```

Listing 4.4: Transmitting BLE Advertising Packets

The following provides a detailed explanation of the key functions: `initializeRadio`, `prepareForBeacon`, and `transmitData`.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

The `initializeRadio` function carries out the following steps:

```
1 void initializeRadio(SX1280Hal &radio)
2 {
3     radio.Reset();
4     radio.Wakeup();
5     radio.IoIrqInit(dio_irq_handler);
6     radio.SetRegulatorMode(USE_DCDC);
7     radio.SetStandby(STDBY_RC);
8 }
```

Listing 4.5: Function to initialize the SX1280 radio settings.

1. **Reset:** The transceiver is reset to ensure it enters the `startup` state. During this phase, the transceiver performs several calibrations and then switches to `STDBY_RC` mode, where it becomes ready to accept commands.
2. **Wakeup:** The `Wakeup` method is invoked to check the current state of the transceiver and ensure it's operational.
3. **Interrupt Initialization:** The pins used for handling interrupts are initialized using the `IoIrqInit` method, allowing the radio to manage interrupt events.
4. **Power Regulator Configuration:** The transceiver's power regulator is set to DC-DC mode, optimizing energy consumption.
5. **Standby Mode:** Finally, the transceiver is placed in standby mode (`STDBY_RC`) to conserve power, with the oscillator configured to run at 13MHz, further reducing power usage.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

`prepareForBeacon` handles the following tasks.

```
1 void prepareForBeacon(SX1280Hal &radio)
2 {
3     radio.SetPacketType(PACKET_TYPE_BLE);
4     radio.SetModulationParams(0x45, 0x01, 0x20);
5     radio.SetPacketParams(0x20, 0x10, 0x04, 0x00, 0x00, 0x00,
6                           0x00);
7     radio.SetBleAdvertiserAccessAddress();
8     uint8_t crcSeed[3] = {0x55, 0x55, 0x55};
9     radio.SetCrcSeed(crcSeed);
10    radio.SetTxParams(10, RADIO_RAMP_02_US);
11    uint16_t irq_mask = 0x0001;
12    radio.SetDioIrqParams(irq_mask, 0x0001, 0x0000, 0x0000);
13 }
```

Listing 4.6: `prepareForBeacon` function implementation for configuring the SX1280 radio for BLE packet transmission

1. **Packet Type Definition:** The packet to be transmitted is defined as a BLE type, according to Table 11-42 of the datasheet [16] using the `SetPacketType` method.
2. **Modulation Parameters Configuration:** The modulation parameters are configured according to Tables 14-17, 14-18, and 14-19 of the datasheet [16]. These values are fixed since each table provides only one option. Table 14-17 sets `modParam1` to `0x45`, corresponding to a bit rate of 1 Mb/s and a bandwidth of 1.2 MHz. Table 14-18 defines `modParam2` with a value of `0x01`, indicating a modulation index of 0.5. Finally, Table 14-19 specifies `modParam3` with a value of `0x20`, associated with a shaping factor (BT) of 0.5.
3. **Packet Parameters Setup:** The packet parameters are set. The `packetParam1` uses the code `0x20` to indicate that the maximum payload length is 31 bytes. The `packetParam2` defines a 3-byte CRC with the code `0x10`. The `packetParam3` does not affect the configuration because no test is being performed. The `packetParam4` enables whitening with the code `0x00`, and the remaining arguments up to 7 are filled with `0x00`, according to tables 14-20, 14-21, and 14-22 of the datasheet [16].

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

4. **Access Address Setting:** The access address is set using the `SetBleAdvertiserAccessAddress` method, which writes the address `0x8E89BED6` into memory address `0x09CF` and the three subsequent memory addresses, as specified in Table 14-26 of the datasheet [16].
5. **CRC Seed Configuration:** The CRC seed value `0x555555` is written to memory address `0x9C7` and the two subsequent addresses, as specified in Table 14-25, using the `SetCRCSeed` method.
6. **Transmission Parameters Setup:** The transmission parameters are configured using the `SetTxParams` method, which takes two arguments: the first corresponds to the output power, `power`, and the second defines the ramp-up time (`rampTime`). The output power, P_{out} , is given by the formula:

$$P_{out} = -18 + power \quad (4.1.1)$$

where the power range varies between -18 dBm and 13 dBm, as indicated in Table 11-47 of the datasheet [16]. The power amplifier ramp-up time is selected using the `rampTime` parameter, which is defined according to Table 11-49 of the datasheet [16].

7. **Interrupts and GPIO Configuration:** The interrupts and GPIOs are configured using the `SetDioIrqParams` method, which accepts three parameters: a mask where each bit corresponds to the desired interrupt, and three additional masks that refer to the three pins. Each bit of the pin mask indicates the interrupt to be monitored. For example, if the “transmission completed” interrupt is to be used (the only relevant one during transmission), the mask `0x0001` should be used. To assign this interrupt to pin DIO1, `0x0001` is applied, while leaving the masks for pins DIO2 and DIO3 set to zero. The corresponding bit for each interrupt can be referenced in Table 11-73 of the datasheet [16].

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

Within the loop, the `transmitData` function is called, which performs the following for each advertising channel

```
1 void transmitData(SX1280Hal &radio, const uint8_t *data,
2   uint8_t header, uint8_t len, uint32_t interval_ms)
3 {
4   for (int i = 0; i < sizeof(channel_table) / sizeof(
5     channel_table[0]); ++i)
6   {
7     radio.SetRfFrequency(channel_table[i].freq_hz);
8     radio.SetWhiteningSeed(channel_table[i].whitening);
9     tx_done = false;
10    radio.SetupAdvPdu(0x80, header, len, data, ctx.
11      bd_addr_le);
12    radio.SetBufferBaseAddresses(0x80, 0x00);
13    radio.SetTx({RADIO_TICK_SIZE_1000_US, 1});
14    while (!tx_done){tight_loop_contents();}
15    printf("TX BLE in channel %d\n", 37 + i);
16    displayPacketStatus(radio);
17    radio.ClearIrqStatus(0x0001);
18    sleep_ms(interval_ms);
19  }
20 }
```

Listing 4.7: Function to Transmit BLE Data Across Channels with Specified Interval

1. **Frequency Selection:** The `SetRfFrequency` method is used to specify the frequency at which the packet will be transmitted.
2. **Whitening Seed:** The whitening seed for that specific frequency is set using the `SetWhiteningSeed` method.
3. **PDU Setup:** The `SetupAdvPdu` method writes the defined PDU header, data length, data, and BLE address into the buffer starting at address `0x80`.
4. **Buffer Management:** The `SetBufferBaseAddress` method is called to indicate which addresses will be used for transmission and which for reception.

4.1. IMPLEMENTATION OF THE BLE STACK FOR BEACON TRANSMISSION

5. **Transmission Mode:** The `SetTx` method is used to enter transmission mode, and a timeout is configured. If this timeout is set to zero, the transceiver will return to `standby` mode once the transmission is complete.
6. **Status Check:** The `displayPacketStatus` function is used to print whether the packet was sent by checking a flag bit returned by the `getPacketStatus` command.
7. **Transmission Interval:** An interval is applied between transmissions on different channels.

Challenges and Solutions

Throughout the development of the Bluetooth Low Energy (BLE) stack for the SX1280 transceiver, several significant challenges were encountered.

1. Selection of Programming Language and Environment

The initial challenge was selecting the appropriate programming language and environment to program the SX1280 chip. Initially, Python was considered for its simplicity and wide use in prototyping. However, difficulties arose regarding compatibility and performance, particularly when attempting to interface with the Raspberry Pi Pico and the SX1280 chip. After thorough evaluation, C++ was chosen as the preferred language due to its performance efficiency and compatibility with the Raspberry Pi Pico framework. Additionally, using Visual Studio Code with the Raspberry Pi extension on a Windows machine provided a seamless development experience, eliminating the need for a Linux virtual machine. This decision streamlined the development process, allowing for direct hardware interaction and better resource management.

2. Chip Initialization and Timing Configuration

The next major challenge was the proper initialization of the SX1280 chip. The complexity of the chip's timing requirements, as outlined in its datasheet, made it difficult to achieve stable initialization. Initially, there were issues with the timing sequences, particularly with the reset procedure at the start, which led to inconsistent behavior of the chip. The breakthrough came with the discovery of Semtech's development code, which included predefined methods that adhered to the precise timing requirements needed for the chip's operation. By integrating these methods, especially the initial reset sequence, the initialization process was stabilized, ensuring reliable performance.

3. Capturing BLE Packets and Validating Transmission

The most significant challenge involved transmitting BLE packets correctly and ensuring their capture by scanning applications. While the spectrum analyzer confirmed that BLE signals were being transmitted, these signals could not be captured by BLE scanning apps, causing considerable confusion and doubt about the implementation's correctness. This problem was eventually traced back to several key factors. First, the configuration of the whitening seed needed to be adjusted for each transmission frequency, as each channel had its specific whitening configuration. The seeds used were obtained from the BTstack library [27]. Second, there were errors in defining the packet fields and utilizing the chip's buffer. The pointer that marked the start of the packet was incorrectly set after the header, when it should have been positioned before the packet header. Lastly, implementing frequency hopping across the three advertising channels was necessary to comply with BLE advertising protocols fully. After addressing these issues—adjusting the whitening seed, correcting the buffer pointer, and enabling frequency hopping—the BLE packets were successfully captured and validated using the scanning applications, confirming that the BLE stack was correctly implemented.

4.2 Integration of BLE and mioty

This section details the integration process of the BLE stack for beacon transmission with the provided software for mioty transmission. The subsection **4.2.1: System Integration Plan** outlines the strategy followed to send advertising beacons every second and mioty packets every 15 minutes, covering aspects such as the inclusion of the aforementioned codes, timing, and energy efficiency. Finally, the subsection **4.2.2: Implementation Steps** reviews the integration from a coding perspective, specifically through a demonstration program.

4.2.1 System Integration Plan

Objective

The first step is to clearly define the objective of integrating the two technologies. As illustrated in the Figure 4.3, the goal is to combine the BLE and mioty modules into a single system. This integrated module should be capable of transmitting advertising beacons every second and mioty packets every 15 minutes, both using a UUID.

4.2. INTEGRATION OF BLE AND MIOTY

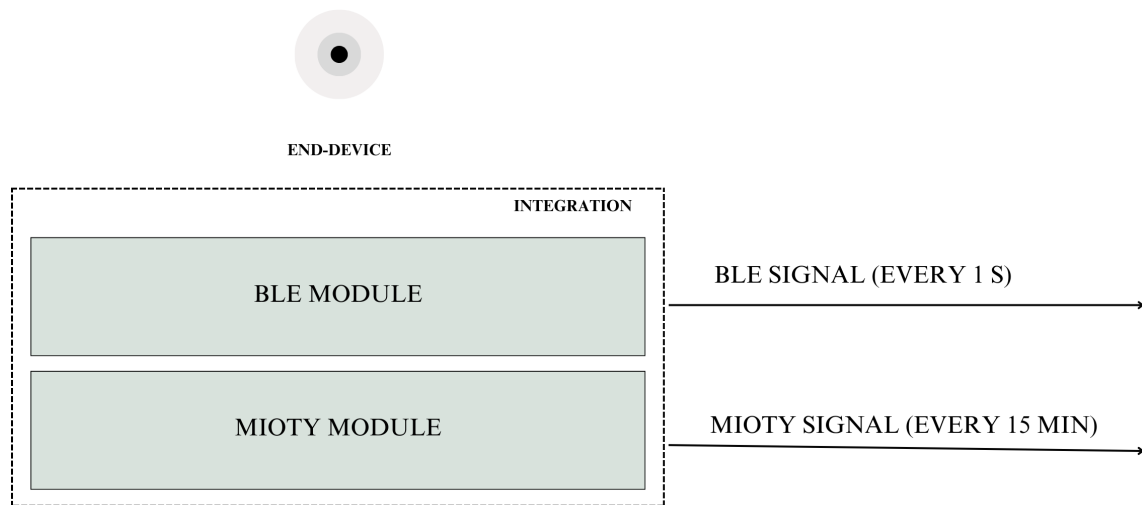


Figure 4.3: Diagram illustrating the integration of the BLE and mioty modules within an end-device.

Considerations for Integration

The second step involves reviewing the key aspects that need to be considered based on the system specifications.

Several important factors must be taken into account for the integration:

- **Energy Efficiency of the SX1280 Chip:** It is crucial to minimize the time the chip spends in the TX state, as this would lead to unnecessary energy consumption.
- **Energy Efficiency of the Code Managing BLE to mioty Transitions:** The transition between BLE and mioty should be managed in a way that minimizes the computational resources required from the microcontroller. This includes optimizing the timing control mechanisms to avoid high energy consumption.
- **Ease of Using the Developed BLE Stack Without Modifying the Original mioty Code:** The integration should be designed in such a way that the BLE stack can be incorporated without altering the provided mioty code. This ensures that the advantages of the modulation are realized without compromising the integrity or functionality of the original mioty implementation.

4.2.2 Implementation Steps

After defining the objective and the requirements for the integration, the following steps were taken to develop the final code that combines both technologies.

Understanding the Provided mioty Code

The mioty codebase is structured into multiple folders and files, each designed to perform specific functions. Some folders focus on technical aspects of the mioty technology, such as encryption and TS-UNB. A crucial folder in this setup is `Trx`, which contains the `SX1280.h` file, similar to the one used in our BLE code. This file defines a class that provides all the methods necessary for transmitting packets.

One approach to integrating BLE functionality could be to modify `SX1280.h` by adding the required BLE methods. However, this would complicate the integration process, as it would necessitate a careful examination of which methods are already implemented and which are not, potentially resulting in a more complex and less modular codebase. Maintaining separate implementations for BLE and mioty could be more beneficial for clarity and modularity.

After reviewing the provided code, the main file, `sendString.cpp`, serves as a demonstration. It utilizes `RPPicoTsUnb.h` to handle SPI initialization between the microcontroller and the radio chip and configure a TS-UNB node using the Fraunhofer TS-UNB library. The program sets up the node, defines the necessary network and radio parameters, and repeatedly sends a message, blinking an LED to indicate successful transmissions.

Interestingly, this program functions similarly to the BLE main program, preparing the mode before transmitting the frame. Thus, a simple integration strategy would be to add the BLE functions directly into the `sendString.cpp` code, requiring only the inclusion of the `UTILS`, `SX1280Driver`, and `TRANSMITTER` folders to the project.

Timing Strategy

Analyzing the `transmitData` function Listing 4.7 from the `TRANSMITTER` module shows that it transmits a BLE packet on each channel with a interval.

In the diagram shown in Figure 4.4, the timing sequence executed by the `transmitData` function during BLE transmissions is illustrated. This function transmits one BLE packet per advertising channel, ensuring that each packet is sent with an interval of one second between them.

4.2. INTEGRATION OF BLE AND MIOTY

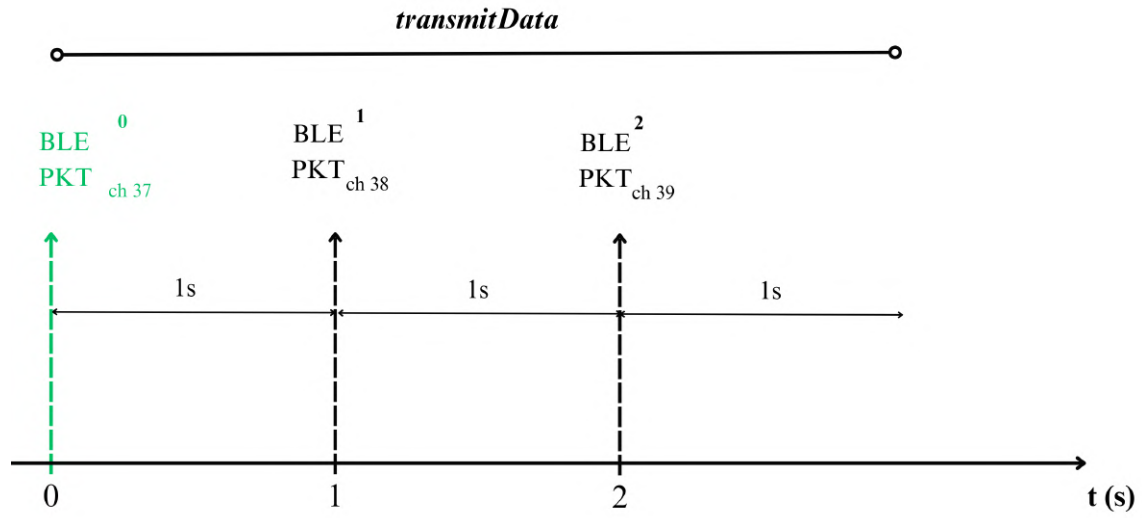


Figure 4.4: Timing diagram followed by `transmitData`

If we consider the timing requirements, BLE packets should be sent every second, while mioty packets need to be transmitted every 15 minutes. One possible approach for implementation is to calculate the number of BLE packets that will be sent before a mioty packet is transmitted. This can be effectively managed using a counter. The calculation is shown below:

$$15 \text{ minutes} \times 60 \text{ seconds/minute} = 900 \text{ seconds} \quad (4.2.1)$$

Since BLE packets are transmitted every second, 900 BLE packets will be sent before a mioty packet is transmitted.

Given that the `transmitData` function sends 3 BLE packets per call, it needs to be called a total of:

$$\frac{900 \text{ BLE packets}}{3 \text{ BLE packets/call}} = 300 \text{ times} \quad (4.2.2)$$

The diagram in Figure 4.5 shows the process where the `transmitData` function sends a BLE packet every second on each advertising channel. Once the counter reaches 299 (indicating 15 minutes), a mioty packet is sent, and the cycle restarts.

4.2. INTEGRATION OF BLE AND MIOTY

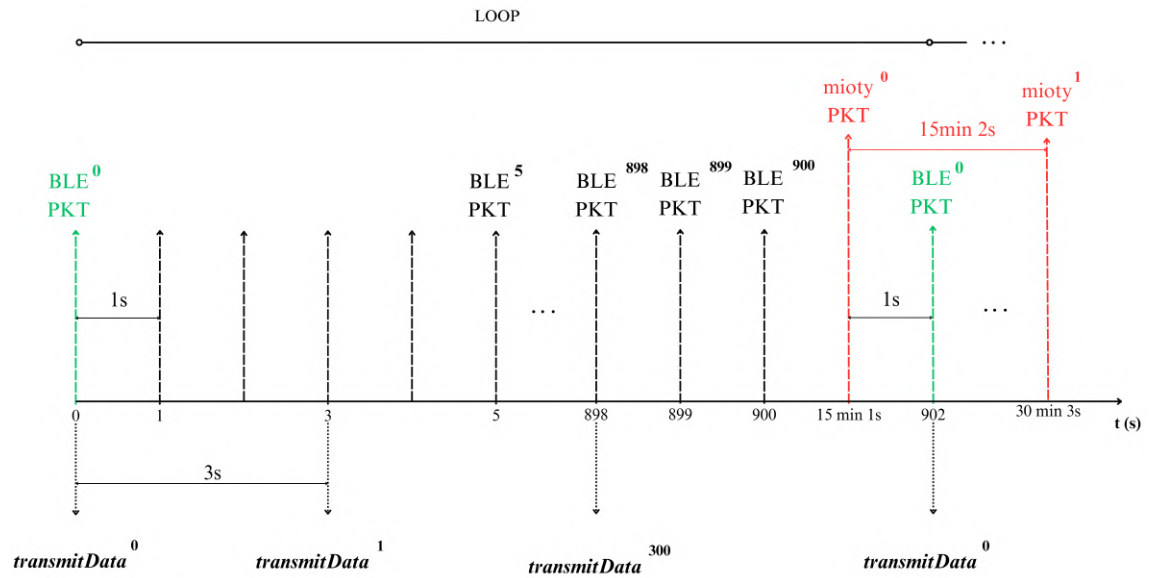


Figure 4.5: Timing Diagram that the integration code would follow

Implementing mioty and BLE Integration in Code

This section presents a code example implementing the strategy of transmitting BLE packets every second and mioty packets after every six BLE transmissions. To achieve this, the `SX1280DRIVER`, `UTILS`, and `TRANSMITTER` folders must be added to the project, ensuring `utils.h` and `utils.cpp` are included in the `utils` folder if it already exists.

The program begins by including the necessary header files.

```

1 #include "pico/stdlib.h"
2 #include "hardware/gpio.h"
3 #include <stdio.h>
4 #include "SX1280Driver/sx1280.h"
5 #include "UTILS/utils.h"
6 #include "Transmitter/transmitter.h"
7 #include "RPPicoTsUnb.h"

```

Listing 4.8: Including Necessary Header Files

4.2. INTEGRATION OF BLE AND MIOTY

Constants and macros are defined.

```
1  #define MAC_NETWORK_KEY 0x53, 0xad, 0xf8, 0x02, 0xb1, 0x97, 0
    xf2, 0x73, 0x8d, 0x5d, 0xdd, 0xa5, 0x77, 0xe1, 0xfa, 0x9c
2  #define MAC_EUI64 0x70, 0xb3, 0xd5, 0x67, 0x70, 0xff, 0x01, 0
    x70
3  #define TRANSMIT_PWR 20
4  #define INTERVAL 100
5  #define BLE_PKTS_BTW_BLEMIOTY 6
6  #define CALLS (BLE_PKTS_BTW_BLEMIOTY / 3)
```

Listing 4.9: Defining Constants and Macros

The program sets up a namespace and initializes the mioty node.

```
1  using namespace TsUnbLib::RPPico;
2  TsUnb_EU1_Lambda80_t TsUnb_Node;
```

Listing 4.10: Initializing the MIOTY Node

The `prepareForMioty` function prepares the mioty node for transmission by initializing it, setting the transmission power, and configuring the network key and MAC address.

```
1  void prepareForMioty()
2  {
3      TsUnb_Node.init();
4      TsUnb_Node.Tx.setTxPower(TRANSMIT_PWR);
5      TsUnb_Node.Mac.setNetworkKey(MAC_NETWORK_KEY);
6      TsUnb_Node.Mac.setAddress(MAC_EUI64);
7      TsUnb_Node.Mac.extPkgCnt = 0x01;
8  }
```

Listing 4.11: Preparing the mioty Node for Transmission

Next, the `uuid` array is defined for BLE and mioty packets. This UUID (Universally Unique Identifier) is used in iBeacon advertisements. Additionally, an array named `ibeacon_data` is created to store the BLE advertising packet.

4.2. INTEGRATION OF BLE AND MIOTY

```
1  uint8_t uuid[16] = {
2      0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
3      0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10
4  };
5
6  uint8_t ibeacon_data[] = {
7      0x02, 0x01, 0x06,
8      0x1A, 0xFF, 0x4C, 0x00,
9      0x02, 0x15,
10     0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
11     0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10,
12     0x00, 0x01, 0x00, 0x02, 0xC5
13  };
14
15  uint8_t header = 0x42;
16  uint8_t len = sizeof(ibeacon_data);
```

Listing 4.12: UUID and iBeacon Data Definition

The `main()` function initializes the radio, prepares it for BLE transmissions, and alternates between BLE and mioty transmissions. First, the standard I/O is initialized for communication with the radio module, and then the `SX1280Hal` object is created to control the SX1280 radio.

```
1  int main()
2  {
3      stdio_init_all();
4      SX1280Hal radio(17, 7, 6, 19, 16, 18);
5      waitUntilUsbConnected();
6      initializeRadio(radio);
7      initializeLed();
8      int calls_counter = 0;
9      prepareForBeacon(radio);
```

Listing 4.13: Main Function Initialization

4.2. INTEGRATION OF BLE AND MIOTY

Inside the main loop, the `transmitData` function is called to send the BLE packet on all three BLE advertising channels. The counter is incremented with each BLE transmission, and once a certain number of transmissions have been made, the program switches to mioty transmission.

```
1  while (true)
2  {
3      transmitData(radio, ibeacon_data, header, len,
4                  INTERVAL);
5      calls_counter++;
6      if (calls_counter >= CALLS)
7      {
8          printf("Switching to Mioty\n");
9          prepareForMioty();
10         char uuid_char[16];
11         for (int i = 0; i < 16; i++)
12         {
13             uuid_char[i] = (char)uuid[i];
14         }
15         TsUnb_Node.send((uint8_t *)uuid_char, sizeof(
16             uuid_char));
17         printf("TX Mioty\n");
18         printf("Switching to BLE\n");
19         sleep_ms(1000);
20         calls_counter = 0;
21         SX1280Hal radio(17, 7, 6, 19, 16, 18);
22         initializeRadio(radio);
23         prepareForBeacon(radio);
24     }
25 }
```

Listing 4.14: Main Loop and Transmission Handling

4.2. INTEGRATION OF BLE AND MIOTY

A timing diagram Figure 4.6 illustrating this process would appear as follows:

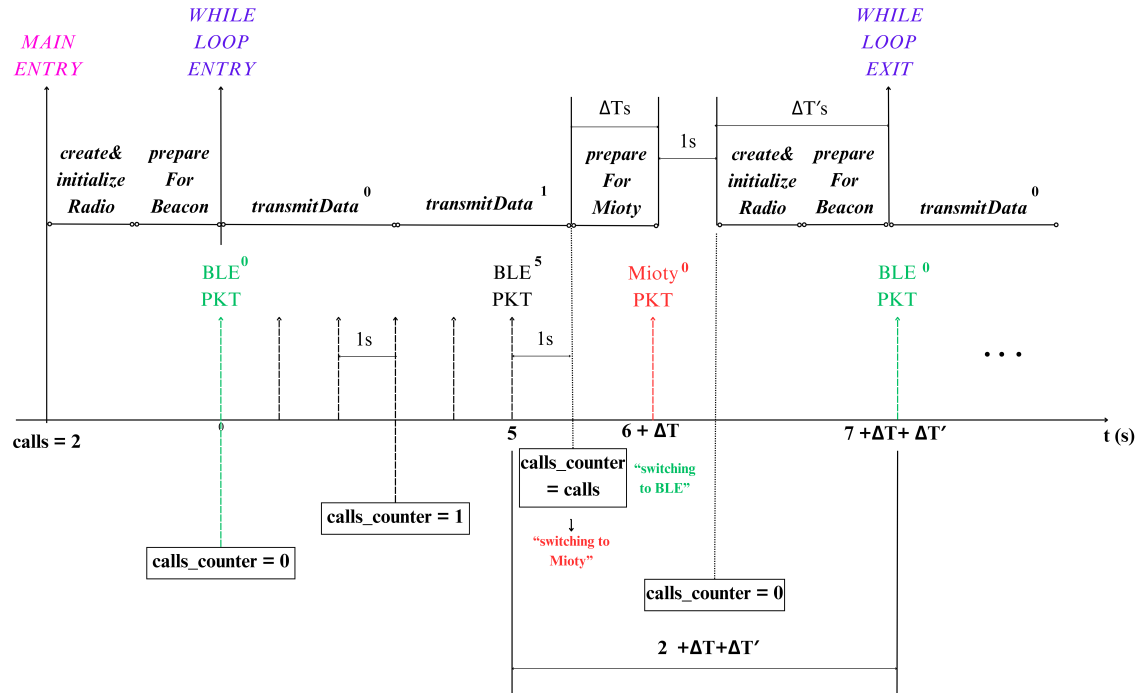


Figure 4.6: Detailed timing diagram of the BLE module, showing the sequence and timing of beacon transmission events.

At the beginning of the main function, an object of the `SX1280Hal` class is created and initialized to prepare for BLE beacon transmission.

The number of `transmitData` calls, denoted as “calls“, is defined by the desired number of BLE packets to be transmitted (if the rate is one packet per second, the packets would correspond to seconds) divided by the fixed number of bursts that occur in the `transmitData` function (which is always 3). This is defined before the `while` loop.

Inside the `while` loop, a counter named `callsCounter` is initialized to zero, and BLE packets are transmitted by calling the `transmitData` function. When the required number of BLE packets has been transmitted, the system prepares for mioty transmission, which takes some time, denoted by ΔT . Once this time has passed, the mioty packet is transmitted, and after one second, the counter is reset. The object of the `SX1280Hal` class is recreated and reinitialized, and the loop continues.

4.3. DEVELOPMENT AND INTEGRATION OF A WEB APPLICATION FOR SYSTEM MANAGEMENT

From the diagram, we can draw the following conclusions:

- **Larger mioty Intervals Increase Counter Value:** The larger the interval between mioty packets, the higher the counter value will be, as more BLE packets are transmitted in the meantime.
- **Calculation of BLE Packets Between MIOTY Transmissions:** The number of BLE packets transmitted between two mioty packets can be calculated using the formula:

$$\text{NumBLEpkts} = T_{\text{mioty}} \times \text{Rate}_{\text{BLE}} \quad (4.2.3)$$

- **Regular Interval for BLE Packets:** The interval between BLE packets is consistently set to 1 second, except during transition periods. During these transitions, there is an additional delay of two seconds, plus the time required to create, initialize, and configure the chip for BLE packet transmission again.
- **Approximate Timing Between mioty Packets:** The actual time between the transmission of two mioty packets might not precisely match the desired interval due to some inherent delays. However, this discrepancy is minimal and generally close to the target interval.

4.3 Development and Integration of a Web Application for System Management

This section focuses on developing the interaction between the components shown in the Figure 4.7, outlined with dashed lines.



Figure 4.7: Part to be developed in section 4.3

4.3.1 Overview of Web Application Development

This section provides an overview of the web application development, including its architecture, database components, the use APIs [17], and the choice of technologies such as MongoDB [28] and Next.js [20].

Architecture Overview

Figure 4.8 illustrates the overall architecture of the system, which consists of several components: the Gateway, the API, the MongoDB Database, and the web application (Web-App). This architecture is designed to efficiently handle the collection and visualization of location data sent from various devices.

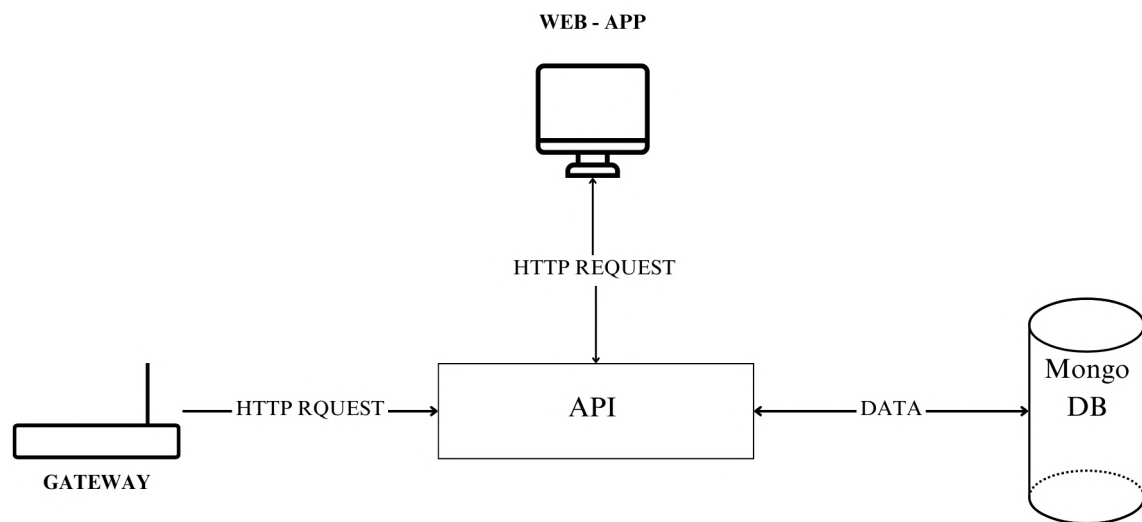


Figure 4.8: Detailed architecture diagram of the web application

The Gateway acts as an intermediary that receives BLE and mioty packets from the end-device, from which it obtains a series of data about the device's location, including coordinates. This data is sent via HTTP requests to the API Server, which is developed using Next.js. The API Server processes these requests and performs CRUD operations (Create, Read, Update, Delete) on the MongoDB Database.

Once the data is stored in the database, the Web-App (also developed with Next.js) makes HTTP requests to the API to fetch updated data that is displayed on the user interface. This allows end-users to view relevant data in real time or at specific times, such as device locations, entry or exit events, and other relevant data.

4.3. DEVELOPMENT AND INTEGRATION OF A WEB APPLICATION FOR SYSTEM MANAGEMENT

Database Structure

In MongoDB [28], data is organized into collections, which are equivalent to tables in SQL databases [29]. In this application, the `locations` collection is utilized to manage location data sent from the Gateway, storing each location update as a JSON [30] document that includes details such as latitude, longitude, device type, and other relevant metadata. Below is an example schema of the `locations` collection:

```
1 {
2   "_id": ObjectId("66d9d05ef455143da1901b1c"),
3   "id": "2E20687F-55AC-42C4-886C-3A82BBACE157",
4   "device": "9B35C9EB-B272-4B28-AFF4-932F858222D7",
5   "device_type": "iOS",
6   "device_model": "iPhone12,1",
7   "timestamp": 1725550686.404767,
8   "longitude": 11.018904255503084,
9   "latitude": 49.58013360894188,
10  "trigger": "enter",
11  "createdAt": "2024-09-05T15:38:06.789+00:00",
12  "updatedAt": "2024-09-05T15:38:06.789+00:00",
13  "__v": 0
14 }
```

Listing 4.15: Example of a MongoDB collection for localitation information

API Endpoints:

The application relies on several API endpoints [21] to interact with the MongoDB database [28]. The main API endpoints include:

- **GET /api/locations:** Retrieves location data from the database based on specified query parameters, such as device ID, timestamp, or geographic coordinates. This endpoint is used to display current and historical location data in the web application.
- **POST /api/locations:** Adds new location data to the database. This endpoint is triggered whenever the Gateway sends new location information, ensuring the database is continuously updated with the latest data.

4.3. DEVELOPMENT AND INTEGRATION OF A WEB APPLICATION FOR SYSTEM MANAGEMENT

Web Application Development

The web application, developed with Next.js, includes components like `AddBeaconModal`, `BeaconDetails`, `BeaconsLists`, `Header`, `Map`, and `SkeletonLoader`. These components are organized into containers that dynamically interact with the API to fetch and display data, ensuring a responsive and interactive user interface.

4.3.2 Development Process

This section describes the development process of the web application, from the initial project setup to the implementation of the API and database design.

Project Initialization with Next.js:

To begin development, a new project was created using Next.js, a React-based framework [31] that enables server-side rendering (SSR) and static site generation (SSG). The initial steps included setting up the development environment and installing the necessary dependencies. The project was initialized using the following command: `npx create-next-app project-name`. This command automatically sets up a development environment with the basic folder and file structure of a Next.js project [32].

API Folders/Files Structure

A folder named `api` was created inside the project's `app` directory. Within the `api` folder, a subfolder named `locations` was generated, containing the `route.ts` file. This file manages the API routes related to location management.

Connecting to the Database

To connect the application to a database, a sign-up was performed on the *MongoDB* platform [33]. An organization and a project were subsequently created within the *Projects* section. For initial testing, a cluster was configured using the free tier option. During the setup, user credentials (username and password) were generated for later use in the connection. The appropriate connection method was selected, and the required package was installed in the project using the command `npm install mongodb`. A `.env` file was created in the project's directory to securely store the `MONGODB_URI` connection string. To ensure proper access, the *Allow access from anywhere* option was configured in the *Network Access* section of the MongoDB platform. In the project, a folder named `lib` was added containing the `db.ts` file, which handles the database connection.

Building Relational Data Models

An additional folder named `models` was created inside the `lib` directory, containing the file `location.ts`. These files define the data schemas for the locations, using objects that describe the structure and properties of each entity in the database.

Building the API

The `route.ts` file within the `locations` folder was modified to implement the necessary endpoints for the application. These endpoints allow for the creation, reading, updating, and deletion of location data. The API functionality was thoroughly tested using *Postman* to ensure that each endpoint responded correctly.

- **GET Endpoint:** Retrieves documents from the database based on URL query parameters, returning matching results in JSON format.
- **POST Endpoint:** Adds new documents to the database by processing data from an HTTP POST request and responding with a success message and the saved data.

Web Interface Development

This section details the development of the main components of the “Beacon Tracker” application’s web interface. Each component has been designed to optimize the user experience, offering an intuitive and efficient interface for managing and visualizing beacon data.

Web Interface Components:

- **Header (`Header.tsx`):** This component provides a responsive header for the “Beacon Tracker” application. It uses *Tailwind CSS* [34] to maintain a clean and functional design, highlighting the application title.
- **Beacons List (`BeaconsList.tsx`):** The `BeaconsList.tsx` component displays an interactive list of available beacons, allowing the user to select a specific beacon to view the location. Designed with *Tailwind CSS*, this component is responsive and facilitates the display of important details such as device type, model, geographical coordinates, and timestamp. This enhances navigation and usability within the application.
- **Interactive Map (`SimpleMap.tsx`):** The `SimpleMap.tsx` component utilizes a Google Maps `iframe` [35] to display the real-time location of the selected beacon.

4.4. TESTING AND VALIDATION

Figure 4.9 displays the *Beacon Tracker* interface with an interactive map on the left showing beacon locations and a user-friendly beacon list on the right for efficient device management and tracking.

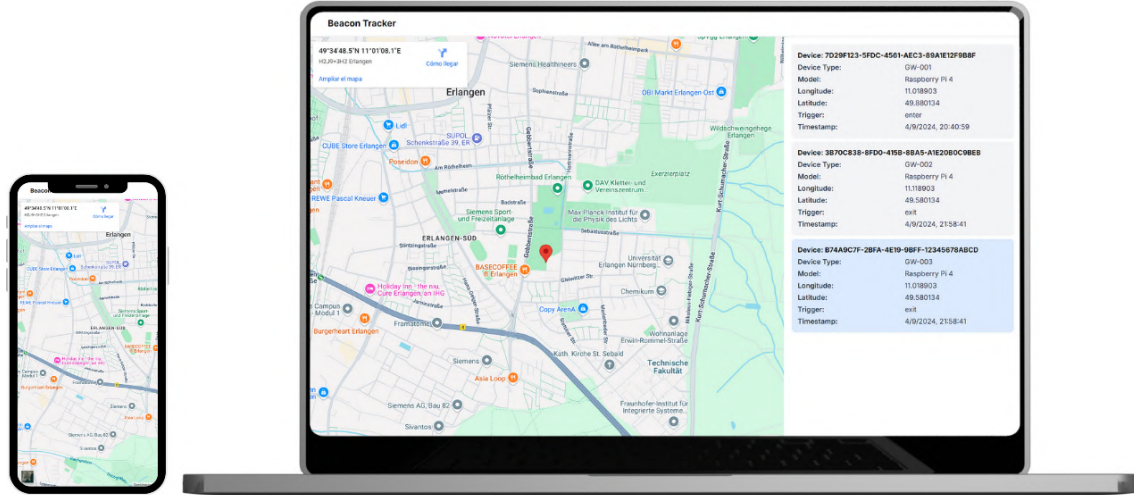


Figure 4.9: Web interface of the *Beacon Tracker* application showing the interactive map and the list of available beacons.

4.4 Testing and Validation

To verify the system, a series of tests were conducted to assess both the transmission of BLE and mioty signals, as well as the visualization of location data on the web interface.

Verification of Radio Transmission

To ensure the SX1280 chip was correctly configured for BLE signal transmission, an IZT R3301 spectrum analyzer [36] was employed to monitor the 2.4 GHz ISM band.

The transmission status was further verified using the `GetPacketStatus` command.

Validation of Packet Content

To validate that BLE packets were correctly formatted and contained valid data, mobile applications like *nRF Connect* [37] was used to capture and analyze Bluetooth traffic emitted by the SX1280 chip.

4.4. TESTING AND VALIDATION

System Integration Testing with Simulated Gateway

To evaluate the interaction between the end-device, gateway, server, and web application, the *Locative* app [38] was used to simulate a gateway environment. The app allows for beacon registration and HTTP notifications upon detection, simulating the gateway's role in receiving BLE signals, calculating coordinates, and forwarding this information to the server. The following steps were conducted:

1. Download the *Locative* app on an iPhone.
2. Register a beacon within the app, ensuring the fields align with those defined in the iBeacon configuration, as shown in 4.10.
3. Configure the app to send HTTP POST requests to a specified URL upon beacon detection.
4. Set up the API to handle incoming data from the app correctly.
5. Monitor server logs to verify the receipt and processing of the POST requests.
6. Confirm that new entries are created in the database with the information provided by the app.
7. Ensure the web interface accurately displays the location information transmitted by the mobile device acting as a gateway.

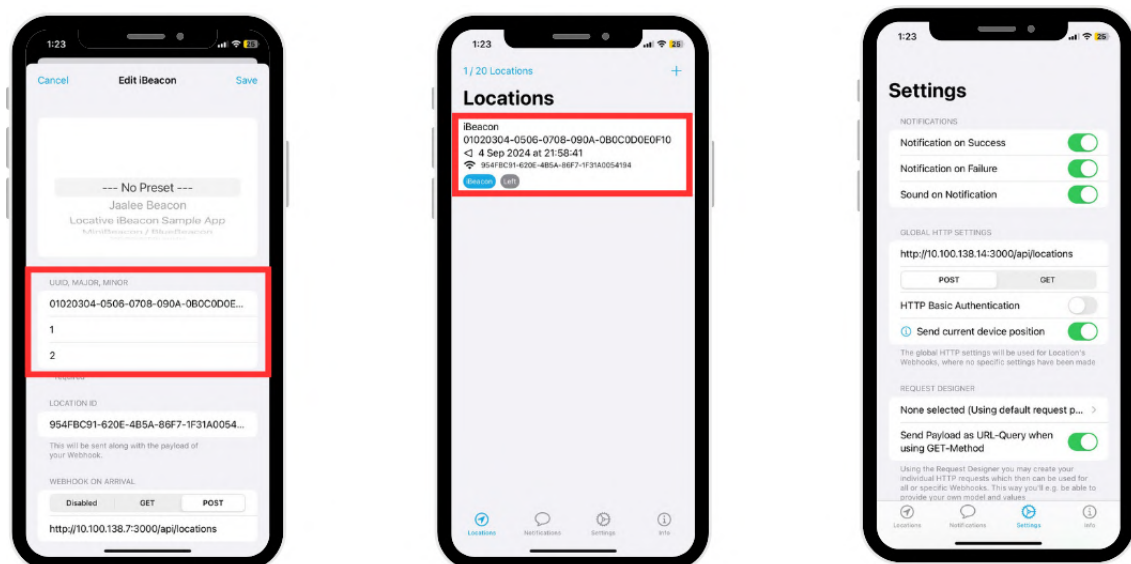


Figure 4.10: *Locative* interface with the fields to be modified highlighted in red.

4.4. TESTING AND VALIDATION

Further Testing with mioty Integration

Additional tests were performed to verify system performance when integrating mioty with BLE technology. The same validation steps were applied, focusing on ensuring proper timing, verifying dual-technology transmissions using the spectrum analyzer, and repeating the gateway simulation using the *Locative* app.

5 Results and Analysis

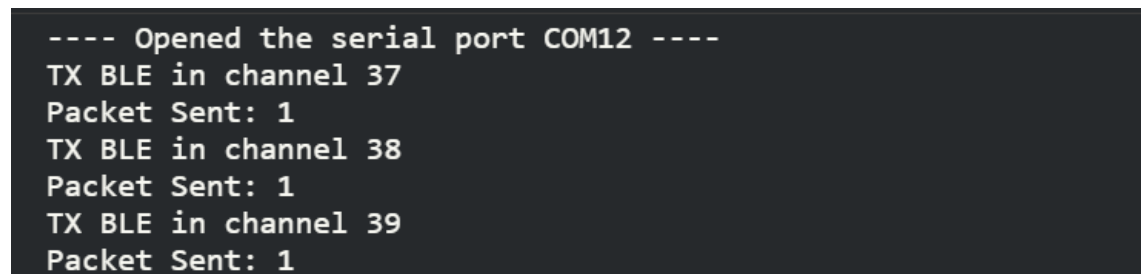
This chapter presents the outcomes of the system validations discussed in the previous section. Each validation step is explained in detail to highlight the key results of the implementation.

5.1 System Implementation Results

Radio Transmission Verification

The radio transmission was successfully verified. By using the `GetPacketStatus` command, the transmission status was accurately checked through the flag bit, as detailed in Table 11-67: *Status Packet Status Byte* [16]. This confirmed that, firstly, the communication between the Raspberry Pi Pico and the transceiver was functioning correctly, and secondly, that the configuration of the SX1280 radio was properly applied.

Figure 5.1 shows the debug message obtained when executing the `transmitData` function 4.7.



```
---- Opened the serial port COM12 ----
TX BLE in channel 37
Packet Sent: 1
TX BLE in channel 38
Packet Sent: 1
TX BLE in channel 39
Packet Sent: 1
```

Figure 5.1: Serial window showing the successful transmission of three BLE packets on different channels.

However, the debug messages indicating "packet sent" were not sufficient to ensure complete system functionality. A spectrum analysis was necessary to confirm that the radio was transmitting on the correct frequencies. The results were as follows:

5.1. SYSTEM IMPLEMENTATION RESULTS

First, the radio signals from the SX1280 chip were successfully visualized on the spectrum analyzer, as shown in Figure 5.2. In the image, two green columns are visible, each consisting of horizontal lines representing the frequency spectrum of the received signals. As the time between signals increases, the gap between the lines also grows. The column on the right displays signals transmitted on the first advertising channel frequency, 2.402 GHz. The column on the left corresponds to signals from the SX1280 chip, configured to transmit on 2.4 GHz frequency, allowing its signals to be distinguished from those of other devices.

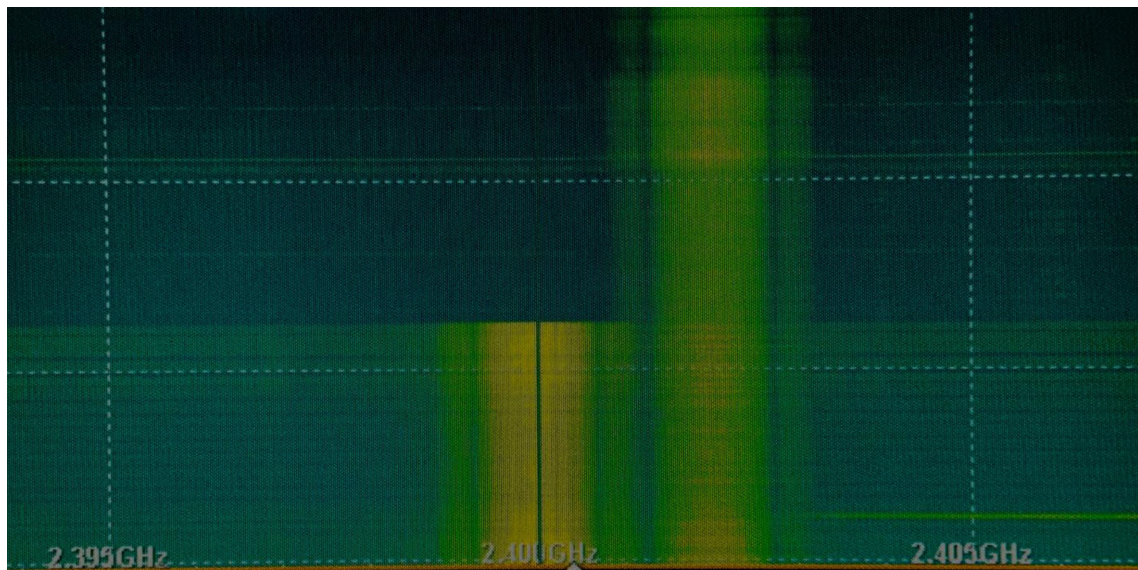


Figure 5.2: BLE packets transmitted at specified intervals on a frequency close to the advertising channel.

Additionally, Figure 5.3 illustrates two modulated signals. The one on the left is centered at 2.4 GHz and reaches a higher amplitude due to the proximity of the chip to the antenna receiver. The signal on the right, centered at 2.402 GHz, corresponds to the advertising channel.

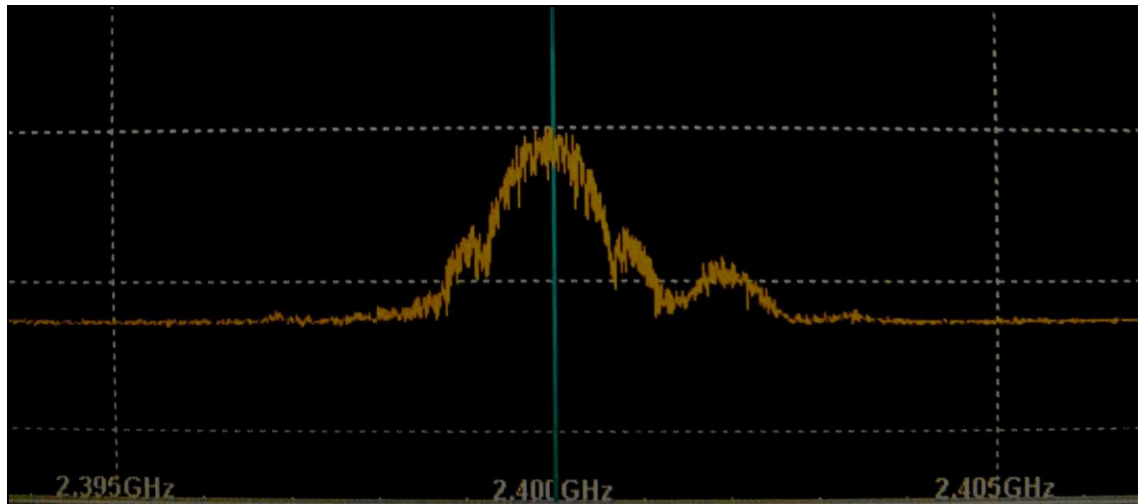


Figure 5.3: Modulated signal of a BLE packet transmission near the advertising channel.

Packet Content Validation

While the previous results confirmed that the radio transmission was functioning, they did not validate whether the fields configured in the code were applied correctly.

The nRF Connect application [37], installed on an Android device, was used to capture and analyze BLE packets. The application displayed the advertising fields, payload, and metadata, including signal strength and packet type, as defined in the code. This provided thorough validation, ensuring that the transmitted packets adhered to the expected BLE format and contained the correct UUIDs and other relevant information. Figure 5.4 demonstrates the captured BLE packet, confirming that the SX1280 was properly configured for BLE communication.

5.1. SYSTEM IMPLEMENTATION RESULTS

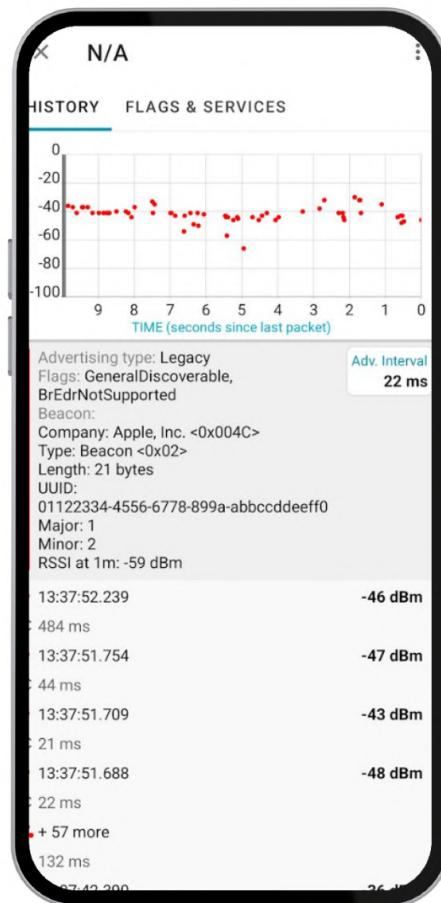


Figure 5.4: nRF Connect application capturing a BLE packet.

System Integration Testing with Simulated Gateway

Although the previous results confirmed the BLE stack's functionality for advertising packet transmission, they only validated part of the system. To test full system integration, including BLE and gateway communication, the Locative application [38] was used to simulate a gateway environment.

5.1. SYSTEM IMPLEMENTATION RESULTS

The application successfully notified the mobile device upon receiving a BLE packet, demonstrating that the SX1280 chip was transmitting BLE packets with fields matching those registered in the app. Figure 5.5 shows the notification generated when the BLE packet was detected.



Figure 5.5: Notification generated upon detecting the BLE packet from the chip.

Furthermore, the Next.js terminal captured the expected POST request from the application after detecting the registered device, confirming the application's configuration. Figure 5.6 presents the POST request message received by the server.

```
> beacon-app@0.1.0 dev
> next dev

▲ Next.js 14.2.5
- Local:      http://localhost:3000
- Environments: .env

✓ Starting...
✓ Ready in 4.6s
  ○ Compiling /api/locations ...
✓ Compiled /api/locations in 768ms (56 modules)
Connected
POST /api/locations 201 in 6015ms
```

Figure 5.6: POST message from the application.

5.1. SYSTEM IMPLEMENTATION RESULTS

Once the request was confirmed, a new entry corresponding to the transmitted data was detected in the MongoDB database, verifying that both the server and database configurations were correct. Figure 5.7 shows the new entry stored in MongoDB, which includes details about the receiver device, such as `device_type` (iOS), `device_model` (iPhone12,1), and `device_id`. The entry also stores the device's location coordinates (`longitude` and `latitude`), the timestamp of the detection, and the trigger event, labeled as "enter". The creation and update timestamps (`createdAt` and `updatedAt`) record when the data was stored in the database.

```
_id: ObjectId('66d9d05ef455143da1901b1c')
id: "2E20687F-55AC-42C4-886C-3A82BBACE157"
device: "9B35C9EB-B272-4B28-AFF4-932F858222D7"
device_type: "iOS"
device_model: "iPhone12,1"
timestamp: 1725550686.404767
longitude: 11.018904255503084
latitude: 49.58013360894188
trigger: "enter"
"
createdAt: 2024-09-05T15:38:06.789+00:00
updatedAt: 2024-09-05T15:38:06.789+00:00
__v: 0
```

Figure 5.7: New entry generated in the MongoDB database.

Finally, an element appeared on the web interface displaying the captured information, confirming end-to-end system functionality. Figure 5.8 shows the transmitted data and the device's location, detected by the BLE packet.

Figure 5.8 shows, on the right side within a blue rectangle, the information transmitted by the Locative application and stored in the MongoDB database. On the left side, it displays the visualization of the device's location, which detected the BLE packet, using the coordinates.

5.1. SYSTEM IMPLEMENTATION RESULTS

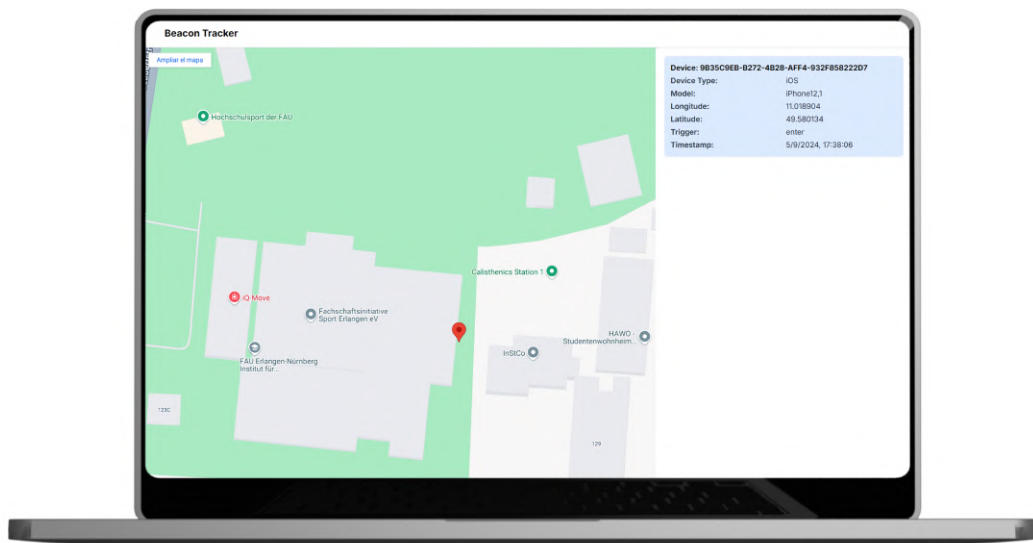


Figure 5.8: Location visualization on the web interface.

Further Testing with Mioty Integration

While the previous results demonstrated that the system was functioning using only the BLE stack for advertising packet transmission, further testing was needed to evaluate the integration of BLE with Mioty. The results obtained using the integrated code for BLE and Mioty transmission were as follows:

Upon integrating Mioty with the BLE stack, the serial terminal (Figure 5.9) displayed debug messages confirming that the timing conformed to the expected sequence shown in Figure 4.6.

```
---- Opened the serial port COM12 ----
TX BLE in channel 37
Packet Sent: 1
TX BLE in channel 38
Packet Sent: 1
TX BLE in channel 39
Packet Sent: 1
TX BLE in channel 37
Packet Sent: 1
TX BLE in channel 38
Packet Sent: 1
TX BLE in channel 39
Packet Sent: 1
Switching to Mioty
TX Mioty
Switching to BLE
TX BLE in channel 37
Packet Sent: 1
```

Figure 5.9: Serial window showing the correct sequence for BLE and Mioty transmissions.

5.1. SYSTEM IMPLEMENTATION RESULTS

Additionally, both BLE and Mioty transmissions were observed on the spectrum analyzer, as depicted in Figures 5.10 and 5.11. Figure 5.10 shows, at the bottom, against a green background, a series of horizontal lines in yellow and red tones, indicating the frequency spectrum generated by the BLE packets transmitted on the first advertising channel frequency.

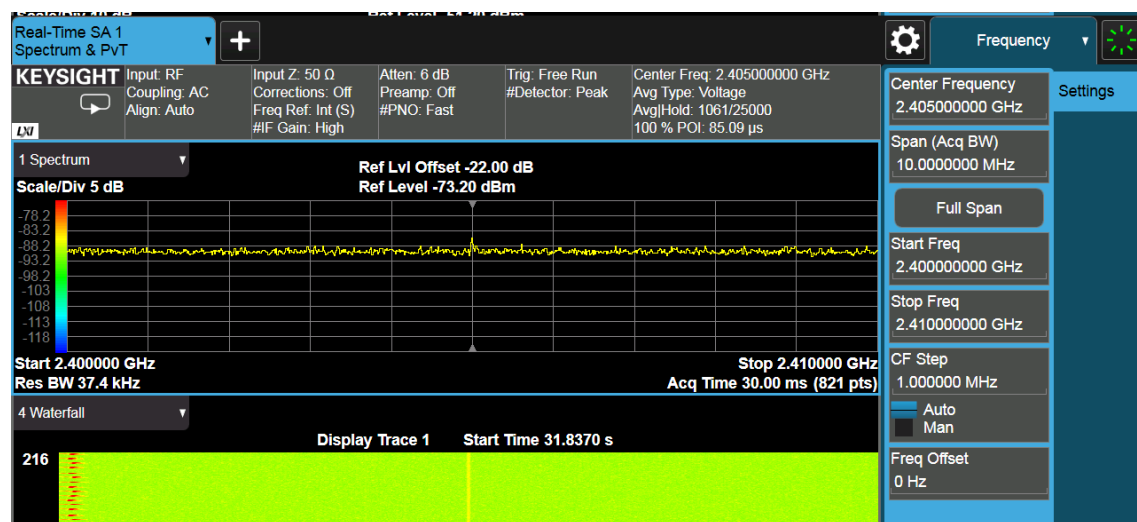


Figure 5.10: Mioty and BLE transmissions observed on the spectrum analyzer.

Figure 5.11 shows, at the bottom, against a green background, a series of horizontal bands representing the frequency spectrum generated by the Mioty transmission. Each band corresponds to a fragment of the packet, and the displacement of the bands indicates that each packet is transmitted on different frequencies following a defined pattern.

5.1. SYSTEM IMPLEMENTATION RESULTS

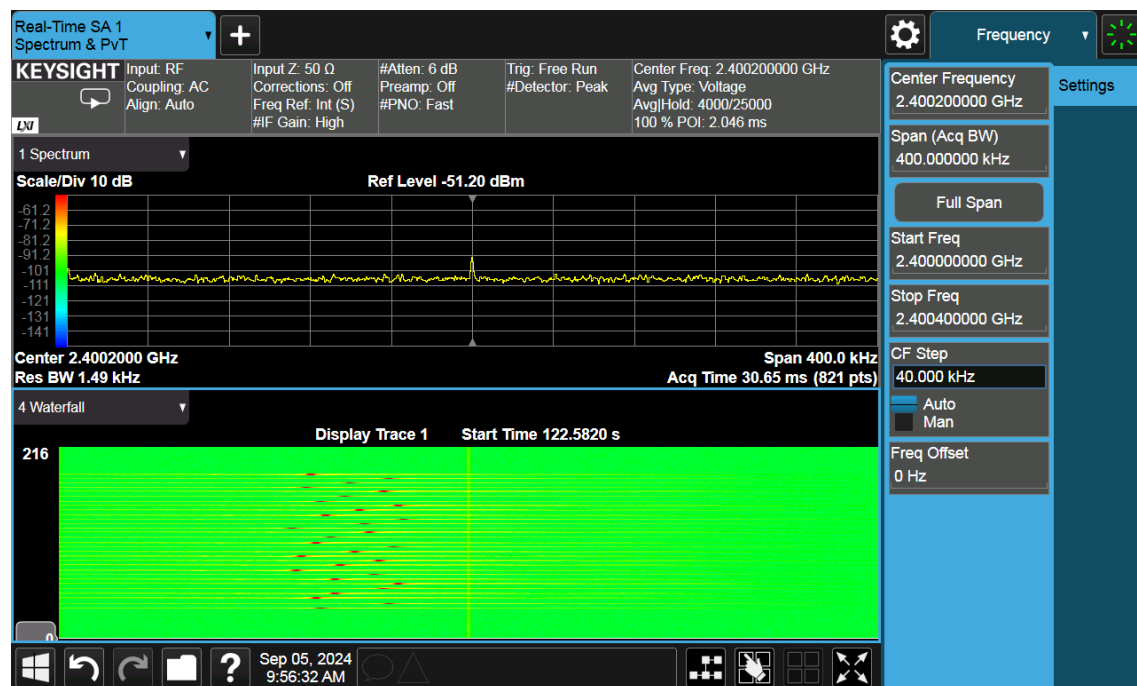


Figure 5.11: Mioty transmission visualized on the spectrum analyzer.

Finally, the BLE packets were once again captured using the nRF Connect application, and the gateway simulation with the Locative application was repeated, achieving the same successful results. This confirmed the system's reliability and consistency when integrating multiple communication technologies.

6 Conclusion and Future Work

This chapter presents the main conclusions of the research conducted, interprets the results obtained, discusses the study's limitations, offers recommendations for future work, and analyzes the broader implications for the field.

6.1 Key Findings

Throughout this study, several key findings have been obtained, effectively addressing the initial objectives and research questions:

- A comprehensive understanding of the advantages and limitations of BLE and mioty technologies was achieved. Additionally, the study highlighted the need for their integration to overcome the limitations of existing asset tracking systems, such as the Apple AirTag, which relies solely on BLE.
- A flexible stack capable of transmitting BLE advertising beacons in both iBeacon and Eddystone formats was developed, enabling communication across different standards.
- The successful integration of this BLE stack with mioty transmission software was accomplished, taking into account factors such as ease of use and energy efficiency. This provides a robust solution for both short-range and long-range communication.
- Finally, a working demonstrator that efficiently utilizes both technologies in terms of energy consumption was successfully implemented and validated through a practical case study.

6.2 Interpretation of Results

The results obtained in this study should be interpreted within the context of the theoretical framework and previous research. The successful transmission and reception of BLE signals, along with the integration of mioty, suggest that the proposed system is not only viable but also offers a robust solution for managing communications in complex environments.

These findings support the theory that combining different communication technologies can enhance the operational efficiency and accuracy of localization systems. Furthermore, the practical results indicate that the developed system has potential for real-time applications, which could revolutionize tracking and monitoring across multiple industries.

6.3 Limitations of the Study

Despite the positive findings, this study has several limitations that should be considered.

Although a complete system is proposed, it does not implement a gateway that receives packets from both technologies and calculates the coordinates based on the proximity of the device. Therefore, throughout this study, it is assumed that the necessary information for calculating coordinates is available.

Additionally, there is no exhaustive study in terms of the power and energy consumption that the transmitting device would have.

6.4 Recommendations for Future Work

Based on the identified limitations and findings of this study, several recommendations for future work are proposed.

Firstly, a decision-making algorithm is proposed to optimize location accuracy and energy efficiency. It selects BLE for short-range scenarios when available, prioritizing precision, and defaults to mioty for long-range communication. The algorithm dynamically adapts to the best signal based on proximity, ensuring efficient and accurate localization.

6.5. IMPLICATIONS FOR THE FIELD

Secondly, it is recommended to explore new applications of the system in different fields, such as fleet management or environmental monitoring, where the combination of BLE and mioty could offer significant advantages.

Lastly, additional functionalities could be added to the web application. For example, the ability to display all available locations simultaneously without needing to select each one from the list to view its location. Functions could also be implemented to filter by specific fields or even the ability to trace on the map the path an asset has followed, thanks to the registration of its entries.

6.5 Implications for the Field

The results of this study have significant theoretical and practical implications for the field of wireless communications and localization systems. Theoretically, this work contributes to the existing body of knowledge by demonstrating that it is possible to effectively combine disparate communication technologies to improve the accuracy and efficiency of systems. Practically, these findings could guide the development of new tracking and monitoring systems that use multiple technologies to provide more comprehensive and reliable solutions.

6.6 Final Thoughts

In conclusion, this study has demonstrated the feasibility of integrating BLE and mioty communication technologies into a cohesive system for real-time tracking and monitoring, from configuration through commands between a chip and a transceiver to the transmission of radio signals, culminating in the visualization of the location by the end-user through the development of an API, a database, and a web interface. Despite the limitations encountered, the results obtained are promising and suggest multiple directions for future research. The development of this system represents a step towards advancing communication technologies and their application in complex environments. Future research will continue to explore these possibilities, further expanding the impact of this work in the field of wireless communications.

List of Figures

2.1	BLE spectrum diagram showing the distribution of advertising and data channels. Adapted from [8], with modifications.	4
2.2	Link Layer Packet Format in BLE. Adapted from [7], with modifications.	5
2.3	Structure of BLE PDU formats, highlighting the PDU header and payload components based on the information provided in the [7].	6
2.4	iBeacon format showing the structure of the UUID is based on the information provided in the [12].	7
2.5	Eddystone UID format showing the structure used for broadcasting unique identifiers for asset tracking based on the Eddystone UID format, as described in the [13].	7
2.6	Flowchart illustrating the operation of Apple AirTag within the Find My network based on the information provided in the [14].	8
2.7	Subdivision of an uplink radio frame into a core frame and an extension frame from [4].	10
2.8	Uplink payload structure on the physical layer from [4].	10
3.1	Hybrid location system with end devices, BLE range, mioty range, and gateways.	13
3.2	System Architecture of the Hybrid Location System Integrating BLE and mioty Technologies.	14
3.3	Raspberry Pi Pico microcontroller interconnected with the SX1280 chip	16
4.1	Diagram of the BLE module, showing the structure and components used.	22
4.2	Diagram of the BLE module, showing the structure and components used.	24
4.3	Diagram illustrating the integration of the BLE and mioty modules within an end-device.	33
4.4	Timing diagram followed by <code>transmitData</code>	35
4.5	Timing Diagram that the integration code would follow	36
4.6	Detailed timing diagram of the BLE module, showing the sequence and timing of beacon transmission events.	40
4.7	Part to be developed in section 4.3	41
4.8	Detailed architecture diagram of the web application	42

4.9	Web interface of the <i>Beacon Tracker</i> application showing the interactive map and the list of available beacons.	46
4.10	<i>Locative</i> interface with the fields to be modified highlighted in red. . . .	47
5.1	Serial window showing the successful transmission of three BLE packets on different channels.	49
5.2	BLE packets transmitted at specified intervals on a frequency close to the advertising channel.	50
5.3	Modulated signal of a BLE packet transmission near the advertising channel.	51
5.4	nRF Connect application capturing a BLE packet.	52
5.5	Notification generated upon detecting the BLE packet from the chip. . . .	53
5.6	POST message from the application.	53
5.7	New entry generated in the MongoDB database.	54
5.8	Location visualization on the web interface.	55
5.9	Serial window showing the correct sequence for BLE and Mioty transmissions.	55
5.10	Mioty and BLE transmissions observed on the spectrum analyzer.	56
5.11	Mioty transmission visualized on the spectrum analyzer.	57

6 Bibliography

- [1] C. Green. “The transformative power of iot in asset tracking”. Accessed on September 16, 2024. (2023), [Online]. Available: <https://www.rtinsights.com/revolutionising-industries-the-transformative-power-of-iot-in-asset-tracking/>.
- [2] Apple Inc., *Airtag*, <https://www.apple.com/airtag/>, Accessed: 2024-08-24, 2021.
- [3] Tech.co. “What is ble tracking? bluetooth low energy tech explained”. Accessed on September 16, 2024. (2023), [Online]. Available: <https://tech.co/asset-tracking/ble-tracking>.
- [4] Fraunhofer Institute for Integrated Circuits IIS, *Mioty™ – physical layer technology*, Fraunhofer Institute for Integrated Circuits IIS, Erlangen, Authored by Dr. Gerd Kilian, Josef Bernhard, Prof. Dr. Wolfgang Koch, Nguyen Quynh Nhu Ho. Accessed: 2024-08-24., 2018.
- [5] D. Cauquil. “Nis summer school iot ble workshop”. Accessed on September 16, 2024. (2018), [Online]. Available: <https://nis-summer-school.enisa.europa.eu/2018/cources/IOT/nis-summer-school-damien-cauquil-BLE-workshop.pdf>.
- [6] Argenox. “Ble advertising primer”. Accessed on September 16, 2024. (2023), [Online]. Available: <https://www.argenox.com/library/bluetooth-low-energy/ble-advertising-primer/>.
- [7] Bluetooth Special Interest Group (SIG), *Bluetooth core specification 5.4*, <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>, Accessed: 2024-08-24. See page 2683 for packet format details., 2023.
- [8] S. Networks. “Ble advertising channels”. Accessed on August 22, 2024. (2024), [Online]. Available: <https://semfionetworks.com/blog/ble-advertisment-channels/>.

-
- [9] A. Devices. “Gaussian frequency-shift keying (gfsk)”. Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.analog.com/en/resources/glossary/gfsk.html>.
- [10] Bluetooth Special Interest Group (SIG), *Bluetooth core specification 5.4*, <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>, Accessed: 2024-08-24. See page 2747 for error checking details., 2023.
- [11] MOKOSmart. “Types of beacons”. Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.mokosmart.com/types-of-beacons/>.
- [12] A. Mbed. “Ble beacons: Uribeacon, altbeacons, ibeacon”. Accessed on September 16, 2024. (2024), [Online]. Available: <https://os.mbed.com/blog/entry/BLE-Beacons-URIBeacon-AltBeacons-iBeacon/>.
- [13] Google. “Eddystone protocol specification”. Accessed on September 16, 2024. (2024), [Online]. Available: <https://github.com/google/eddytone/blob/master/protocol-specification.md>.
- [14] A. Inc. “Apple introduces airtag”. Accessed on September 16, 2024. (2021), [Online]. Available: <https://www.apple.com/newsroom/2021/04/apple-introduces-airtag/>.
- [15] Miromico. “Mioty technology overview”. Accessed on September 16, 2024. (2024), [Online]. Available: [https://miromico.ch/en/technologies/mioty#:~:text=mioty%20is%20a%20Low%20Power,Multiple%20Access%20\(TSMA\)%20technology](https://miromico.ch/en/technologies/mioty#:~:text=mioty%20is%20a%20Low%20Power,Multiple%20Access%20(TSMA)%20technology).
- [16] Semtech Corporation, *Sx1280 datasheet*, Accessed: 2024-08-22. The datasheet provides detailed technical information about the SX1280 chip, including specifications and operating modes., 2024. [Online]. Available: https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/3n000000190Z/Kw7ZeYZuAZW3Q4A3R_IUjhYQCQEJxkuLrUgl_GNNhuUo.
- [17] Cloudflare. “What is an api call?” Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.cloudflare.com/es-es/learning/security/api/what-is-api-call/>.
- [18] R. P. Foundation. “Raspberry pi pico”. Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-pico/>.

-
- [19] Raspberry Pi Ltd, *Getting started with raspberry pi pico-series*, Accessed: 2024-08-22. Available at <https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf>, Raspberry Pi Ltd, Cambridge, UK, 2024.
- [20] Vercel. "Next.js - the react framework for production". Accessed on September 16, 2024. (2024), [Online]. Available: <https://nextjs.org/>.
- [21] TechTarget. "What is an api endpoint?" Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.techtarget.com/searchapparchitecture/definition/API-endpoint#:~:text=API%20endpoints%20serve%20as%20the,Proper%20functioning%20of%20software.>
- [22] Semtech Corporation, *Sx1280 development kit*, <https://www.semtech.com/products/wireless-rf/lora-connect/sx1280>, 2024.
- [23] ScienceDirect. "Bluetooth stack". Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/bluetooth-stack>.
- [24] N. Bits. "Bluetooth low energy protocol stack layers". Accessed on September 16, 2024. (2024), [Online]. Available: <https://novelbits.io/bluetooth-low-energy-protocol-stack-layers/>.
- [25] Microsoft. "Visual studio code - code editing. redefined." Accessed on September 16, 2024. (2024), [Online]. Available: <https://code.visualstudio.com/>.
- [26] R. P. Foundation. "Raspberry pi pico - visual studio code setup". Accessed on September 16, 2024. (2024), [Online]. Available: <https://github.com/raspberrypi/pico-vscode>.
- [27] B. GmbH, *Btstack library for sx1280 chipset*, https://github.com/bluekitchen/btstack/blob/master/chipset/sx128x/ll_sx1280.c, Accessed: 2024-09-24. Refer to the source code for implementation details., 2024.
- [28] M. Inc. "Why use mongodb?" Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.mongodb.com/resources/products/fundamentals/why-use-mongodb#:~:text=MongoDB%20is%20a%20document%20database%20built%20on%20a%20horizontal%20scale,following%20in%20the%20developer%20community.>

-
- [29] SolarWinds. "What is a sql database?" Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.solarwinds.com/resources/it-glossary/sql-database>.
- [30] W3Schools. "Javascript json introduction". Accessed on September 16, 2024. (2024), [Online]. Available: https://www.w3schools.com/js/js_json_intro.asp.
- [31] I. Meta Platforms. "React: A javascript library for building user interfaces". Accessed on September 16, 2024. (2024), [Online]. Available: <https://es.react.dev/>.
- [32] Vercel. "Learn: Create a next.js app". Accessed on September 16, 2024. (2024), [Online]. Available: <https://nextjs.org/learn-pages-router/basics/create-nextjs-app/setup>.
- [33] M. Inc. "Mongodb: The developer data platform". Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.mongodb.com/>.
- [34] T. Labs. "Tailwind css: Rapidly build modern websites without ever leaving your html". Accessed on September 16, 2024. (2024), [Online]. Available: <https://tailwindcss.com/>.
- [35] K. Osei. "Google map iframe with tailwind css". Accessed on September 16, 2024. (2024), [Online]. Available: <https://tailwindflex.com/@kofi-osei/google-map-iframe-with-tailwind>.
- [36] I. Labs. "Izt r3301 - high performance receiver". Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.izt-labs.de/receivers-izt-r3301/>.
- [37] N. Semiconductor. "Nrf connect for mobile". Accessed on September 16, 2024. (2024), [Online]. Available: <https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile>.
- [38] Locative. "Locative: Geofence and beacon". Accessed on September 16, 2024. (2024), [Online]. Available: <https://apps.apple.com/de/app/locative-geofence-und-beacon/id725198453>.